

**UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ**

LUCRARE DE LICENȚĂ

Sistem de Management și Interceptare a Traficului de Rețea (Firewall)

**Conducător științific
Conf. univ. Dr. Bufnea Darius**

*Absolvent
Cuciurean Emilian-Petruț*

2026

ABSTRACT

In recent years, the Internet has grown significantly, increasing browsing speed, task volumes and the amount of content provided. This growth expands the attack surface, raising the risk of encountering malicious actors whose actions can have irreversible impacts, leading to the compromise of personal information or digital assets.

A fundamental solution to reduce these risks is the development of firewalls. A Firewall acts as a security barrier, monitoring and controlling incoming and outgoing traffic based on predefined security rules, blocking unauthorized access and regulating data flow between trusted internal networks and potentially dangerous external sources. Beyond simple packet filtering, modern firewalls perform stateful inspection, tracking the state of active connections to distinguish between legitimate responses and unsolicited traffic. They serve as the first line of defense in a layered security strategy, preventing threats before they reach the system. By isolating sensitive resources and analyzing every byte of data that crosses the network perimeter, firewalls provide necessary visibility and control to maintain digital stability and security.

To address these evolving security needs, this thesis presents CuciSec, a firewall software system, built around a two-layer architecture that separates the brute, rapid flow of network packets from the detailed analysis and decision based on rules and detectors. The first layer consists of an nftables ruleset operating in kernel space, which enforces a multi-stage filtering pipeline: persistent blacklist drops, LAN whitelist bypass, rate limits against volumetric attacks, and selective traffic to userspace via the NFQUEUE mechanism. The second layer, userspace, receives packets through the NetfilterQueue library and processes each one through four detection stages in fixed priority order. First stage evaluates administrator-defined rules, followed by second stage that applies an angloalgorithm for detecting volumetric attacks in an interval of time. Third stage is for honeypot detection, which treats any connection to trap ports as malicious, and the final stage, for deep packet threats which matches HTTP payloads against vulnerable signatures. When a threat is detected, the source IP is banned and injected into blacklist tables, within the nftables and a database. Database writes are decoupled from packet processing through an asynchronous Producer-Consumer queue, eliminating I/O latency on the verdict path. The system is managed through a FastAPI REST backend and a React web application, which expose rule management, event logs, blacklist control, and real-time traffic statistics aggregated from both the database and live nftables counters.

Cuprins

1	Introducere	4
1.1	Contextul securității cibernetice	4
1.2	Evoluția amenințărilor cibernetice	5
1.3	Motivație și obiectivele proiectului	8
1.4	Contribuții originale aduse domeniului	9
1.5	Structura lucrării	10
1.6	Utilizarea instrumentelor IA generative	10
2	Concepte fundamentale în Securitatea Rețelelor	11
2.1	Transmiterea datelor prin rețea	11
2.1.1	Modelul OSI	11
2.1.2	Modelul TCP/IP	13
2.1.3	Encapsularea și decapsularea datelor	14
2.2	Tehnologia Firewall	14
2.2.1	Mecanisme de translatare a adreselor	16
2.2.2	Sisteme de monitorizare și protecție a fluxului de date	17
2.2.3	Segmentarea rețelei și principiul apărării stratificate	19
2.2.4	Firewall-uri de generație următoare (NGFW)	20
3	Interceptarea și Procesarea Pachetelor	22
3.1	Istoricul sistemelor de filtrare în Linux	22
3.2	Subsistemul Netfilter	23
3.2.1	Arhitectura nftables	24
3.2.2	Comunicarea kernel-userspace	26
3.3	Soluții alternative	27
3.3.1	Windows Filtering Platform (WFP)	27
3.3.2	Packet Filter (PF) pentru macOS	29
4	Dezvoltarea Sistemului CuciSec	31
4.1	Arhitectura sistemului	31
4.2	Stratul kernel-Space	32

4.3	Stratul userspace	33
4.4	Server și interfața web	35
4.5	Evaluarea sistemului	38
5	Concluzii și Direcții Viitoare de Dezvoltare	41
5.1	Sinteza realizărilor și a contribuțiilor aduse	41
5.2	Limitări ale sistemului	41
5.3	Direcții viitoare de extindere	42
	Bibliografie	44

Capitolul 1

Introducere

1.1 Contextul securității cibernetice

Securitatea cibernetică (*Cybersecurity*) este domeniul IT care se ocupă cu prevenirea daunelor, protejarea și restaurarea computerelor, a sistemelor și a serviciilor de comunicații, a comunicațiilor prin cablu și electronice, inclusiv a informațiilor conținute de acestea, pentru a asigura disponibilitatea, integritatea, autentificarea, confidențialitatea și nerepudiarea acestora [Nat26]. Acest domeniu acoperă o arie largă: rețele, servere, aplicații, infrastructuri cloud și orice sistem conectat la internet. Problema centrală pe care o adresează este că datele și serviciile digitale traversează canale publice în care orice actor rău intenționat poate intercepta, modifica sau bloca comunicațiile.

Obiectivele pe care această ramură o urmărește sunt cunoscute sub denumirea de triada CIA (*Confidentiality, Integrity, Availability*) [Sta17].

Confidențialitatea: Confidențialitatea datelor (*Data confidentiality*): asigură că persoanele neautorizate nu au acces la informațiile private; *Intimitatea* (*Privacy*): asigură controlul asupra modulului în care sunt stocate, colectate și manipulate informațiile personale.

Integritatea: Integritatea datelor (*Data integrity*): asigură că datele și programele sunt modificate numai într-un mod specificat și autorizat; *Integritatea sistemului* (*System integrity*): asigură modul în care un sistem își îndeplinește atribuțiile într-un mod neperturbat de către o persoană din exterior.

Disponibilitatea: Disponibilitatea datelor (*Availability*) garantează că utilizatorii autorizați au acces la date și resurse atunci când au nevoie de acestea.

1.2 Evoluția amenințărilor cibernetice

O amenințare cibernetică (*cyber threat*) reprezintă un potențial pericol de a afecta, de a perturba, sau a de fura date din sistemele informatice, prin acces neautorizat, distrugere, sau exploatarea cu succes a unei vulnerabilități. [Nat26]. O vulnerabilitate este o slăbiciune a unui sistem informatic, a procedurilor de securitate a sistemului, a controalelor interne sau a implementării acestuia, care ar putea fi folosită sau declanșată de către o amenințare cibernetică [Nat26]. Un atac este orice tip de activitate rău intenționată care încearcă să colecteze sau să distrugă resursele sistemului informatic sau informațiile în sine [Nat26]. Atacurile pot fi pasive (când atacatorul doar interceptează și observă traficul fără să îl modifice), sau active (când modifică, șterge sau perturbă resursele vizate) [Sta17].

Tipuri de amenințări frecvente:

- *Malware* - software malițios instalat pe dispozitivul victimei fără știrea acesteia, folosit pentru furtul de date, monitorizarea activității sau controlul de la distanță al sistemului;
- *Ransomware* - un tip specific de malware care criptează fișierele victimei și cere o răscumpărare pentru cheia de decriptare;
- *Phishing* - tentative de înșelătorie prin care atacatorul mimează o sursă de încredere (bancă, instituție publică) pentru a obține date de autentificare sau informații financiare;
- *Denial of Service (DoS)* și *Distributed Denial of Service (DDoS)* - inundarea unui server cu un volum mare de trafic care depășește capacitatea sa de răspuns, cu scopul de a face serviciul inaccesibil utilizatorilor legitimi; varianta distribuită folosește multe dispozitive compromise (*botnet*);
- *SQL Injection (SQLi)* - Introducerea de cod SQL malițios în formularele unei aplicații pentru a controla sau accesa neautorizat baza de date;
- *Cross-Site Scripting (XSS)* - injectarea de scripturi JavaScript în pagini web pentru a fura date de sesiune sau pentru a controla contul utilizatorului;
- atacuri asupra lanțului de aprovizionare (*supply chain attacks*) - compromiterea unui furnizor sau partener pentru a obține acces indirect la sistemele organizației țintă;
- atacuri sponsorizate de state (*state-sponsored attacks*) - operațiuni de spionaj, sabotaj sau dezinformare conduse de entități cu sprijin guvernamental, care vizează infrastructuri critice sau procese politice.

- atacuri cu inteligență artificială (*AI-powered attacks*) - utilizarea modelelor AI pentru a automatiza generarea de mesaje de tip phishing personalizate, pentru a detecta vulnerabilități în sisteme sau pentru a adapta malware-ul în timp real la mediile vizate [Wor26]

Primele atacuri documentate, în anii 1970–1980, vizau sisteme locale și se propagau prin dischetă sau prin rețelele universitare, aveau scop strict tehnic sau demonstrativ. Între anii 1990–2000, virușii au evoluat într-o amenințare prioritară, dar fără impact financiar. Criminalitatea cibernetică a început în perioada 2000–2010, perioadă în care atacurile au dobândit o componentă financiară, predominând cele de tip phishing. 2010–2020 au predominat atacurile sponsorizate de state, ransomware și supply chain attacks. Din 2022, odată cu generalizarea llm-urilor, inteligența artificială a devenit o unealtă și pentru atacatori, prin automatizarea și scalarea atacurilor de tip phishing, generarea de malware, și recunoașterea vulnerabilităților. Amenințările dominante din ziua de astăzi rămân ransomware, DDoS, intruziuni, supply chain, state-sponsor [Eur25, Goo26, Sta17]. Tabelul 1.1 sintetizează cele mai semnificative incidente cibernetice recente.

Tabela 1.1: Incidente cibernetice majore la nivel global

Incident	An	Tip	Impact
Stuxnet	2010	Malware	Instalații nucleare iraniene deteriorate fizic; primul atac cu consecințe fizice directe [Sta17].
WannaCry	2017	Ransomware	150 de țări, pierderi peste 4 mld. USD.
SolarWinds	2020	Supply chain	Mii de organizații și agenții guvernamentale SUA compromise.
Change Healthcare	2024	Ransomware	100 milioane de înregistrări medicale expuse [U.S25].
Snowflake Campaign	2024	Furt de credențiale	165 de organizații afectate prin conturi cloud fără MFA [Goo24].
Salt Typhoon	2024	Intruziune	Grup chinez; peste 200 de companii din 80 de țări compromise, inclusiv AT&T și Verizon [Fed25].
PowerSchool	2025	Furt de credențiale	62 milioane de elevi și 9,5 milioane de profesori afectați [Pow25].
Bybit	2025	Phishing	cel mai mare furt cripto din istorie [Kra25].
M&S, Harrods	2025	Ransomware	Perturbări operaționale majore în retailul britanic [Wor26].

Raportul ENISA Threat Landscape 2025, perioada iulie 2024 – iunie 2025, se bazează pe analiza a 4.875 de incidente de securitate cibernetică înregistrate în spațiul european. Distribuția tipurilor de incidente, ilustrată în Figura 1.1, arată că atacurile DDoS reprezintă 76,7% din totalul înregistrărilor, urmat de intruziuni cu 17,8%. În momentul în care este obținut accesul, cel mai utilizat cod malițios este ransomware-ul, având o rată de 83,5%.

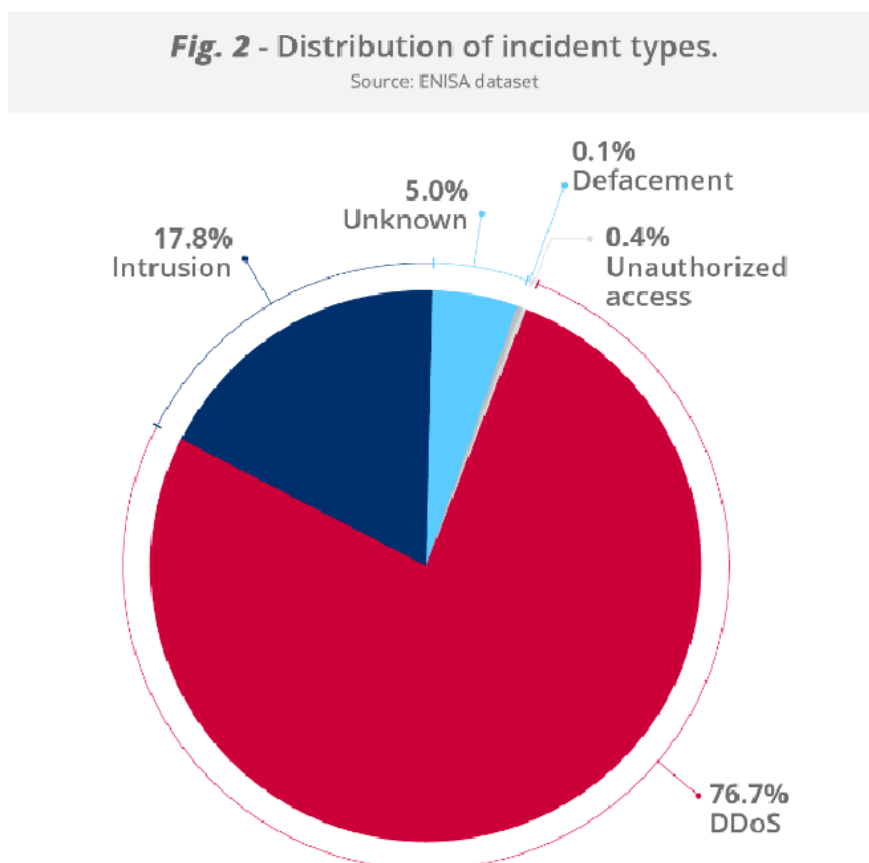


Figura 1.1: Distribuția tipurilor de incidente cibernetice în UE, 2024–2025 [Eur25]

Vectorii de infecție principali sunt prezentați în Figura 1.2. Phishing-ul reprezintă aproximativ 60% din cazurile observate, exploatarea vulnerabilităților 21,3% este al doilea vector ca frecvență, urmat de botneturi 9,9% și malware 8%. Inteligența artificială a accelerat semnificativ eficiența atacurilor de tip phishing: la începutul anului 2025, campaniile de *social engineering* susținute de AI reprezentau peste 80% din activitatea observată la nivel mondial. [Eur25]

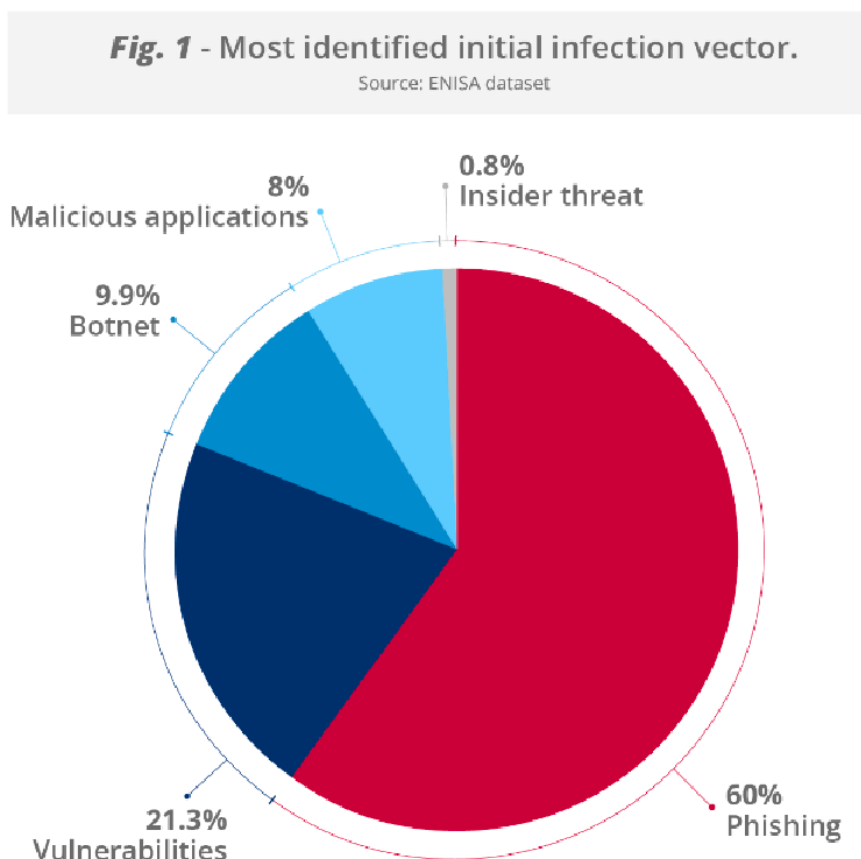


Figura 1.2: Vectorii de intruziune înregistrați în UE, 2024–2025 [Eur25]

1.3 Motivație și obiectivele proiectului

Administrarea clasică a securității la nivel de rețea în sistemele de operare Linux s-a realizat prin utilitare accesibile doar din interfața în linia de comandă (*Command Line Interface CLI*), un exemplu semnificativ fiind *iptables*, utilitar construit peste infrastructura Netfilter. Acest utilitar prezintă o serie de limitări: regulile sunt evaluate secvențial $O(N)$, cu cât lista de reguli este mai lungă, cu atât fiecare pachet necesită mai mult timp pentru a fi procesat, modificarea regulilor necesită reîncărcarea întregului set de reguli, sintaxa este diferită pentru IPv4 și IPv6, ceea ce obligă administratorul să mențină două seturi paralele de reguli pentru aceeași politică de securitate. Faptul că toate transformările se fac din linia de comandă necesită o atenție sporită, întrucât o singură comandă introdusă greșit poate deschide o cale de atac sau poate bloca traficul într-un mod neașteptat. Soluțiile enterprise, Cisco Secure Firewall (Cisco), Strata (Palo Alto Networks) sau Fortigate (Fortinet) rezolvă aceste limitări, oferă interfețe grafice, vizibilitate completă asupra traficului și management mai ușor al deciziilor de securitate, dar sunt costisitoare și nu pot fi inspectate sau modificate de utilizator.

Motivația pentru proiectul CuciSec a fost de a crea un sistem care să depășească

aceste limite, combinând un strat de filtrare brută și rapidă prin *nftables* în kernel cu o logică de inspecție și decizie amănunțită într-un script *Python* și o interfață grafică pentru management și administrare mai prietenoasă utilizatorilor. Sistemul include un motor de reguli configurabile cu prioritate maximă, un subsistem de prevenire a intruziunilor pe trei niveluri independente și un mecanism de blocare reactivă care elimină automat sursa unui atac detectat. Cele trei niveluri de detecție vizează atacurile volumetrice, accesul pe porturi capcană și atacurile care necesită analiză aprofundată a pachetelor. Administrarea regulilor, vizualizarea evenimentelor și monitorizarea statisticilor sunt accesibile într-o aplicație web, fără intervenție directă în linia de comandă.

1.4 Contribuții originale aduse domeniului

Prima contribuție este separarea arhitecturală pe două zone, o zonă de filtrare a fluxului mare de date și o zonă de control aplicată unui sistem de securitate nativ pe Linux: filtrarea pachetelor și menținerea stărilor rămâne în kernel-space folosind utilitarul *nftables*, în timp ce logica de decizională rulează în userspace. O eroare în codul de analiză nu afectează stabilitatea sistemului de operare și nici fluxul de trafic legitim.

A doua contribuție este un motor decizional pe niveluri independente: primul nivel este destinat regulilor statice predefinite de administrator, care permite filtrarea pe baza propriilor cerințe; al doilea nivel dezvoltat este pentru prevenirea atacurilor volumetrice înainte ca acestea să consume resursele de analiză ale nivelurilor superioare; al treilea nivel expune porturi asociate serviciilor vulnerabile și tratează tentativele de conexiune pe aceste porturi drept activitate malițioasă; ultimul nivel dezvoltă un motor *deep packet inspection* care acționează la nivelul aplicație, acesta inspectează conținutul pachetelor pentru a detecta tipare de atac care nu sunt vizibile în antetele de rețea.

A treia contribuție rezolvă problema resetării setului de reguli atunci când este modificat. S-a implementat un mecanism de reîmprospătare automată atunci când o regulă este adăugată, ștearsă sau modificată, menținând setul de reguli persistent atât în kernel cât și într-o bază de date.

A patra contribuție este dezvoltarea unei interfețe grafice care permite manipularea regulilor într-un mod mai simplu, vizualizarea jurnalelor de decizie și a traficului pe o perioadă îndelungată de timp pentru a ușura activitățile de administrare ale specialiștilor de rețele.

1.5 Structura lucrării

Lucrarea este structurată în cinci capitole, organizate după o progresie logică de la fundamentul teoretic la implementarea practică și validarea sistemului după cum urmează:

Capitolul 2 detaliază conceptele teoretice necesare pentru înțelegerea funcționării sistemelor de securitate la nivel de rețea. Acoperă modelul de referință OSI, tehnologiile de filtrare a traficului, sistemele de detecție și prevenire a intruziunilor, strategiile de segmentare a rețelelor bazate pe zone logice, și arhitecturile de apărare pe mai multe straturi.

Capitolul 3 descrie evoluția utilităților de filtrare în Linux, arhitectura subsistemului Netfilter, mecanismul nftables, utilitarul de comunicare Netlink și protocolul NFQUEUE. Deasemenea, sunt menționate și soluțiile disponibile pe sistemele de operare Windows și macOS.

Capitolul 4 prezintă în detaliu arhitectura sistemului, cum s-a implementat filtrarea traficului, analiza și logica de decizie pentru pachetele aflate în tranzit către o rețea care se dorește a fi protejată, cât și aplicația de administrare a sistemului.

Capitolul 5 sintetizează contribuțiile realizate, limitările actuale ale arhitecturii și direcțiile viitoare de extindere.

1.6 Utilizarea instrumentelor IA generative

În timpul elaborării acestei lucrări, autorul a folosit instrumentul bazat pe inteligență artificială generativă Claude (dezvoltat de Anthropic) în scopuri de asistență redacțională, optimizare a stilului academic, sugerare de repere bibliografice și sugestii privind îmbunătățirea unor funcționalități ale proiectului. După utilizarea acestui instrument, autorul a revizuit și editat conținutul generat și își asumă întreaga responsabilitate pentru conținutul lucrării.

Capitolul 2

Concepte fundamentale în Securitatea Rețelelor

2.1 Transmiterea datelor prin rețea

2.1.1 Modelul OSI

Modelul *Open Systems Interconnection* (OSI) a fost definit de *International Organization for Standardization* (ISO) și publicat ca standard internațional în 1984 cu scopul de a crea o arhitectură modulară, pe șapte niveluri, care să permită comunicarea între calculatoare [TW10]. Fiecare mesaj generat de o aplicație traversează aceste straturi descendent la emisie și ascendent la recepție, suferind procese succesive de modificare, așa cum este ilustrat în Figura 2.1.

Nivelul 1. Fizic (*Physical Layer*). Se ocupă de transmiterea biților brute printr-un mediu de comunicație fizic. Acest strat codifică fluxul de biți în semnale adecvate canalului de transmisie (indiferent dacă sunt diferențe de potențial electric pe cabluri de cupru, impulsuri luminoase în fibra optică sau unde electromagnetice în medii wireless) și gestionează specificațiile mecanice, electrice și funcționale ale sistemelor hardware, asigurând sincronizarea la nivel de bit între emițător și receptor.

Nivelul 2. Legătură de date (*Data Link Layer*). Are rolul de a transforma un mediu fizic brut de transmisie într-o legătură sigură între două noduri direct conectate. Acesta organizează biții în unități structurate numite cadre (frames) și implementează mecanisme de control al erorilor (care detectează și gestionează cadrele alterate în tranzit) și mecanisme de control al fluxului (menite să prevină supra-solicitarea unui receptor lent de către un emițător rapid). La acest nivel operează adresele fizice Media Access Control (MAC), echipamente precum switch-urile sau bridge-urile, și protocoale standardizate ca Ethernet și Wi-Fi.

Nivelul 3. Rețea (*Network Layer*). Răspunde de livrarea pachetelor de la o gazdă sursă la o gazdă destinație, traversând rețele multiple și interconectate. Funcția sa

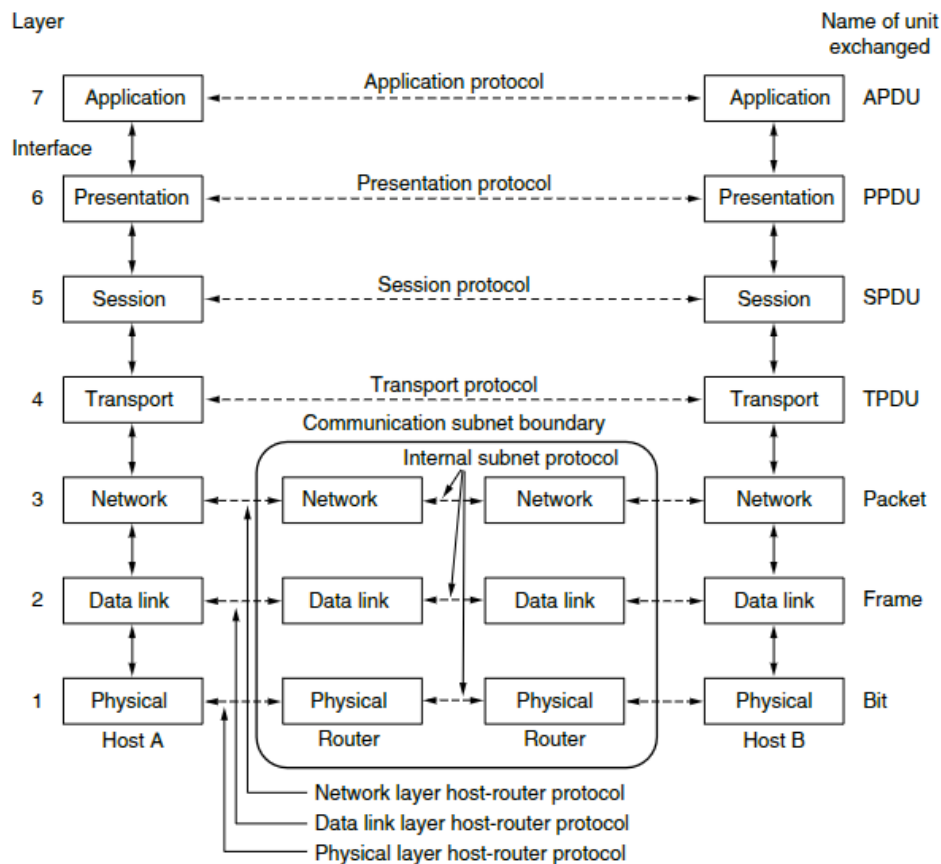


Figura 2.1: Modelul de referință OSI [TW10]

principală este rutarea (determinarea drumului optim prin rețea), realizată de routere pe baza adresării logice globale (adrese IP) [Pos81a]. Câmpurile fundamentale analizate în antetul pachetului sunt adresele sursă și destinație, identificatorul protocolului de nivel superior încapsulat și câmpul Time to Live (TTL), utilizat pentru prevenirea buclilor de rutare.

Nivelul 4. Transport (*Transport Layer*). Nivelul transport asigură livrarea datelor între două gazde identificate prin porturile sursă și destinație, fragmentează datele primite de la aplicație și le reordonează la destinație pe baza numerelor de secvență. Există două protocoale principale: *Transmission Control Protocol* (TCP), care garantează livrarea sigură și detectează pierderile [Pos81b], și *User Datagram Protocol* (UDP), care oferă latență mai mică dar nu garantează integritatea datelor [Pos80].

Nivelul 5. Sesiune (*Session Layer*). Gestionează stabilirea, menținerea și terminarea sesiunilor de comunicație între aplicații. Acesta oferă metode de structurare a dialogului și introduce puncte de verificare (checkpoints) în fluxul de date pentru a permite reluarea transferurilor masive în caz de defecțiune. Protocoalele asociate includ *NetBIOS* și *RPC* (*Remote Procedure Call*).

Nivelul 6. Prezentare (*Presentation Layer*). Se ocupă de sintaxa și semantica

informațiilor transmise, asigurând compatibilitatea între sisteme care pot utiliza reprezentări interne diferite ale datelor. Îndeplinește trei funcții critice: traducerea și formatarea datelor (conversii între seturi de caractere precum ASCII sau Unicode), compresia datelor (reducerea volumului de biți pentru optimizarea lățimii de bandă) și criptarea/decriptarea datelor pentru asigurarea confidențialității. Protocoale de criptare: *Transport Layer Security* (TLS) și *Secure Sockets Layer* (SSL).

Nivelul 7. Aplicație (*Application Layer*). Legătura dintre stiva de protocoale și aplicația utilizatorului. Nu reprezintă aplicația în sine, ci protocolul prin care aceasta comunică: HTTP și HTTPS pentru navigare web, DNS pentru rezoluția numelor, SMTP și IMAP pentru e-mail, FTP pentru transfer de fișiere și SSH pentru administrare și acces de la distanță.

2.1.2 Modelul TCP/IP

Modelul de referință *TCP/IP* își are numele preluat după primele două protocoale de bază și a fost dezvoltat de Departamentul Apărării al Statelor Unite (*Department of Defense* - DoD) în anii '70 ca fundament al rețelei de cercetare ARPANET, precursorul Internetului, care lega sute de universități și sisteme guvernamentale [TW10]. Figura 2.2 prezintă stiva TCP/IP alături de modelul OSI pentru o comparație directă între cele două modele.

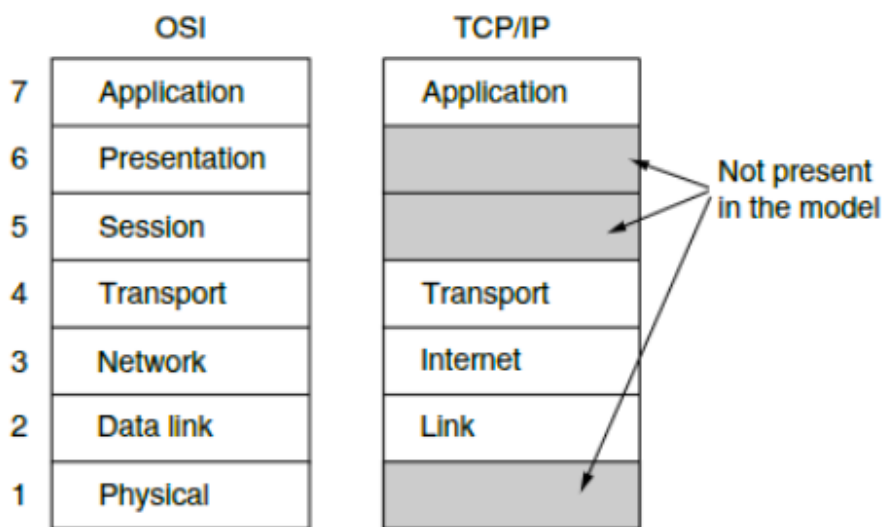


Figura 2.2: Modelele OSI și TCP/IP [TW10]

Cele patru niveluri ale modelului TCP/IP sunt: *Network Access (Link Layer)* care acoperă transmisia fizică și adresarea hardware, echivalent nivelurilor 1 și 2 din OSI; *Internet*, care gestionează rutarea pachetelor prin adrese IP, echivalentul Network Layer; *Transport*, care asigură comunicarea între procese prin protocoalele

TCP și UDP, asemănător modelului OSI; *Application*, care include toate protocoalele de nivel înalt: HTTP, DNS, SMTP, SSH, TLS, corespunzător nivelurilor 5, 6 și 7 din OSI.

TCP/IP este protocolul care rulează efectiv pe internet și pe rețelele moderne. Cu toate acestea, pentru documentarea sistemelor de securitate se folosește în continuare terminologia OSI, din cauza granularității. OSI separă clar Layer 5, Layer 6 și Layer 7, ceea ce permite descrierea precisă a nivelului la care operează un mecanism de securitate.

2.1.3 Encapsularea și decapsularea datelor

Fiecare nivel al stivei OSI adaugă propriile informații înainte de a transmite datele nivelului următor. Unitatea de date a protocolului (*Protocol Data Unit* - PDU) reprezintă structura specifică pe care o iau datele la fiecare nivel, în timp ce conținutul transferat se numește *payload*. Nivelul curent adaugă un antet (*header*) în fața acestui payload. Fiecare nivel are propriul tip de PDU, cu denumire specifică: datele de la nivelele superioare devin *segment* la nivelul 4, *pachet* la nivelul 3 și *cadru* la nivelul 2. La nivelul 1 totul este transmis ca *șiruri de biți* pe mediul fizic. [TW10]

Procesul de encapsulare. Când o aplicație trimite date, acestea pornesc de la Layer 7 și coboară prin stivă spre Layer 1. La nivelul transport, datele aplicației sunt segmentate dacă depășesc dimensiunea maximă admisă și li se adaugă antetul TCP sau UDP, care conține porturile sursă și destinație, numărul de secvență și flagurile de control. La nivelul rețea, segmentul primește antetul IP, cu adresele logice sursă și destinație și identificatorul protocolului de transport. La nivelul data link, pachetul IP este împachetat într-un cadru Ethernet/WI-FI căruia i se adaugă adresele MAC și suma de control (*checksum*) pentru detectarea erorilor de transmisie. *Procesul de decapsulare.* La recepție, gazda destinație execută procesul invers, de jos în sus, de la șir de biți la datele originale [TW10].

2.2 Tehnologia Firewall

Un *firewall* este un device sau program care controlează traficul de date între rețele cu politici de securitate diferite [SH09]. Concret, un firewall se plasează între rețeaua internă (protected network) și cea externă (untrusted network) examinând traficul care intră către rețeaua internă și decide pe baza unui set de reguli predefinit, dacă pachetul este acceptat sau este blocat.

Există mai multe tipuri de firewall, diferențiate prin nivelul la care inspectează traficul și prin capacitățile pe care le oferă.

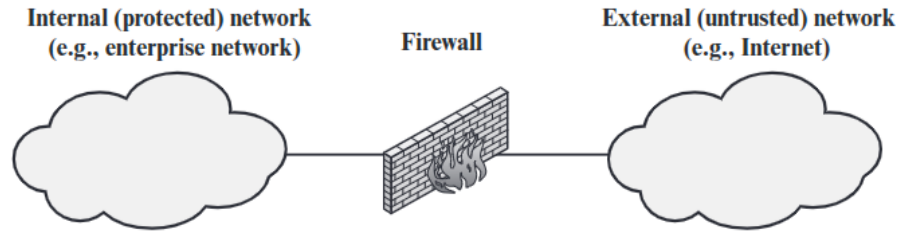


Figura 2.3: Poziționarea unui firewall în rețea, formă generală [Sta17]

Packet Filtering Firewall este cel mai simplu tip de firewall, care urmărește analogia de filtrare *stateless*. Inspectează fiecare pachet individual pe baza câmpurilor din antet: adresele IP sursă și destinație, porturile și protocolul. Decizia se ia independent pentru fiecare pachet, pe baza unui set de reguli, fără să se țină cont de starea conexiunii. În tabelul 2.1 se poate observa o listă de reguli pentru traficul pachetelor de email.

Tabela 2.1: Listă de reguli pentru trafic SMTP [Sta17]

Regulă	Direcție	IP Sursă	IP Dest	Protocol	Port dest.	Acțiune
A	In	External	Internal	TCP	25	Permit
B	Out	Internal	External	TCP	>1023	Permit
C	Out	Internal	External	TCP	25	Permit
D	In	External	Internal	TCP	>1023	Permit
E	Either	Any	Any	Any	Any	Deny

Stateful Inspection Firewall menține o tabelă cu conexiunile active și verifică pentru fiecare pachet contextul sesiunii din care face parte. De exemplu un pachet TCP cu flagul ACK care sosește fără o conexiune anterioară în tabelă este respins automat, chiar dacă adresa și portul sunt permise de reguli. Figura 2.4 prezintă un exemplu al acestui tabel, cu conexiunile urmărite de sistem.

Source Address	Source Port	Destination Address	Destination Port	Connection State
192.168.1.100	1030	210.22.88.29	80	Established
192.168.1.102	1031	216.32.42.123	80	Established
192.168.1.101	1033	173.66.32.122	25	Established
192.168.1.106	1035	177.231.32.12	79	Established
223.43.21.231	1990	192.168.1.6	80	Established
2122.22.123.32	2112	192.168.1.6	80	Established
210.922.212.18	3321	192.168.1.6	80	Established
24.102.32.23	1025	192.168.1.6	80	Established
223.21.22.12	1046	192.168.1.6	80	Established

Figura 2.4: Tabelul de stări a conexiunilor într-un sistem *stateful inspection* [Sta17]

Din punct de vedere al modului de instalare, firewall-urile se clasifică în dispozitive hardware (stații fizice amplasate la perimetrul rețelei), software (instalate pe servere sau hardware) și cloud (*Firewall-as-a-Service*).

2.2.1 Mecanisme de traducare a adreselor

Network Address Translation (NAT) operează la nivelul routerului și mapează adresele IP între rețeaua privată și cea publică [ES01]. Spațiul de adrese IPv4 este limitat la aproximativ 4,3 miliarde de adrese unice și care s-a dovedit insuficient pe măsură ce numărul dispozitivelor conectate la internet a crescut. Adresele private nu sunt rutabile pe internet, deci orice pachet cu adresă sursă privată este ignorat de routerele publice. NAT rezolvă această problemă, se realizează o corespondență 1:1, o adresă privată este mapată la o adresă publică fixă.

Network Address and Port Translation (NAPT) extinde acest mecanism prin modificarea suplimentară a porturilor de la nivelul Transport [ES01]. Mai multe dispozitive private cu adrese diferite partajează o singură adresă publică, fiecare conexiune fiind identificată printr-un port unic. Când un pachet părăsește rețeaua internă, routerul înlocuiește adresa sursă privată cu adresa sa publică și atribuie un port nou din spațiul propriu, recalculând sumele de control din antetele IP și TCP / UDP pentru a păstra validitatea pachetului.

Tabela 2.2: Comparație NAT versus NAPT

	Traducare	IP privată	IP public	Raport
NAT	IP	192.168.1.10	93.1.8.7	1 → 1
NAPT	IP + port	192.168.1.10:1025 192.168.1.11:3343	93.1.8.7:2000 93.1.8.7:2001	2 → 1

Pentru ca NATP să funcționeze, routerul trebuie să rețină corespondența dintre conexiunile active din interior și porturile externe alocate. Această informație este stocată în tabelul de conexiuni, aceeași structură folosită de mecanismul stateful inspection [SH09]. Când sosește un pachet de răspuns din exterior cu portul destinație 2000, routerul consultă tabelul, identifică conexiunea originală și rescrie câmpurile de destinație pentru a ruta pachetul corect înapoi la adresa privată 192.168.1.10:1025. Figura 2.5 ilustrează acest flux: adrese interne distincte trimit trafic simultan, iar routerul le mapează pe o singură adresă publică cu porturi diferite.

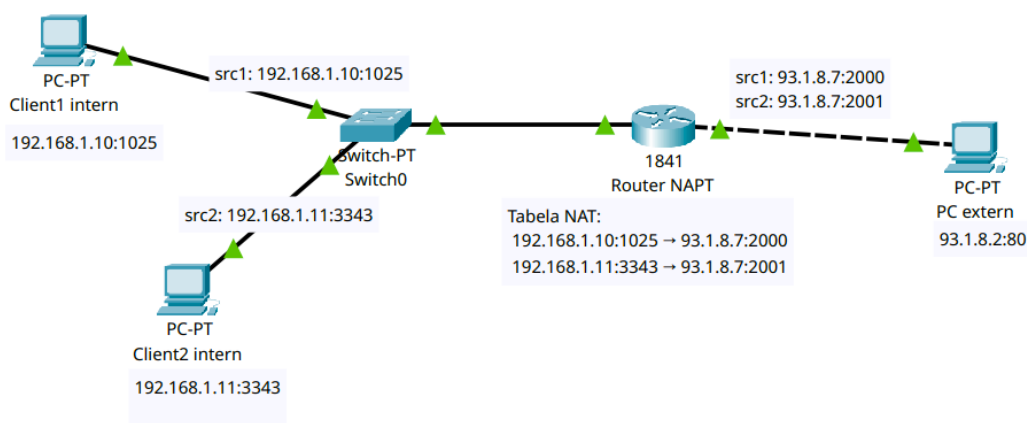


Figura 2.5: Flux NATP cu doi clienți interni în Cisco Packet Tracer. Ambele conexiuni ies pe aceeași adresă publică 93.1.8.7, dar porturi distincte.

Există două variante de traducere în funcție de direcția traficului. *Source NAT* (SNAT) modifică adresa sursă a pachetelor care ies din rețeaua internă spre internet (*masquerade*). *Destination NAT* (DNAT) modifică adresa destinație a pachetelor care intră din exterior și se aplică înainte de routare.

Pentru IPv6 nu mai este nevoie de NAT/NAPT deoarece are spațiu de adresare pe 128 de biți, care oferă aproximativ 3.4×10^{38} adrese unice [DH17].

2.2.2 Sisteme de monitorizare și protecție a fluxului de date

Sistemele de tip firewall controlează accesul la perimetrul rețelei, dar nu sunt proiectate să analizeze în detaliu traficul admis. O conexiune legitimă poate transporta pe parcursul sesiunii activitate malițioasă care nu a fost anticipată în setul de reguli. Din această cauză, s-a inclus un al doilea nivel de securitate: sisteme de detecție și sisteme de prevenție a intruziunilor.

Sistemele de Detecție a Intruziunilor (*Intrusion Detection System IDS*) funcționează în mod pasiv, analizând o copie a traficului de rețea fără să intervină în fluxul real al pachetelor (*out-of-band*). Copia este obținută printr-un port de tip *SPAN* (*port mirroring*) configurat pe switch, care direcționează pachetele duplicate către sistemul

IDS fără să perturbe comunicația originală. Dacă sistemul identifică activitate suspectă, generează alerte, înregistrează evenimentul și notifică administratorul. Traficul rețelei continuă să circule indiferent de decizia sistemului. Consecința directă este că un IDS nu poate bloca un atac în desfășurare, ci semnalează imediat după ce acesta a avut loc.

Un Sistem de Prevenție a Intruziunilor (*Intrusion Prevention System IPS*) este plasat *inline*, imediat după firewall, unde fiecare pachet traversează sistemul IPS înainte de a ajunge la destinație [SH09]. Această poziționare permite sistemului să ia decizii active fără intervenție manuală: să elimine pachetele malițioase, să reseteze conexiunea sau să blocheze dinamic adresa sursă. Această arhitectură introduce însă două riscuri specifice. Primul este latența: fiecare pachet trebuie procesat înainte de a fi transmis mai departe, iar un motor de inspecție lent devine un *bottleneck*. Al doilea este cel de *single point of failure*: dacă sistemul IPS cedează, traficul legitim poate fi întrerupt complet. Comportamentul în caz de defecțiune se configurează explicit prin politica de *fail-open* sau *fail-closed* [Sta17]. *Fail-open* menține traficul activ chiar dacă sistemul de inspecție nu mai funcționează, prioritizând disponibilitatea, iar *Fail-closed* blochează tot traficul la căderea sistemului, prioritizând securitatea.

Tabelul 2.3 sintetizează diferențele operaționale dintre cele două sisteme.

Tabela 2.3: Comparație între IDS și IPS

Criteriu	IDS	IPS
Mod de operare	Pasiv, analizează o copie a traficului	Activ, analizează traficul real
Poziție în rețea	Out-of-band	Inline
Impact trafic	Cedarea sistemului nu afectează traficul	Cedarea sistemului poate bloca traficul
Acțiuni posibile	Alertare, logare, notificare	Drop pachet, resetare conexiune, banare IP sursă

Sisteme bazate pe gazdă (HIDS/HIPS). Un sistem *Host-Based* operează direct la nivelul sistemului de operare al dispozitivului protejat și are acces direct la apeluri de sistem (*syscalls*), procese, modificări de fișiere și traficul local necriptat, fiind precis în identificarea anomaliilor. Dezavantajele sunt consumul ridicat de resurse și necesitatea administrării unui agent pe fiecare nod din infrastructură. În plus dacă sistemul de operare este compromis, un atacator cu privilegii de administrator poate întrerupe funcționarea sistemului.

Sisteme bazate pe rețea (NIDS/NIPS). Un sistem *Network-Based* interceptează și monitorizează traficul pe un segment de rețea fără instalarea în prealabil a unui software. Prin această abordare se asigură protecția centralizată pe mai multe dispozitive, fără consum ridicat de resurse. Limitarea acestuia este incapacitatea de a

inspecta payload-ul traficului criptat.

Apărarea în adâncime. Principiul *Defense in Depth* recomandă utilizarea mai multor straturi de protecție independente, la niveluri diferite, analizând traficul în mod secvențial. Fiecare strat este conceput pentru combaterea unui anumit tip de amenințare, această abordare micșorează șansele unui atac cu succes întrucât dacă un strat nu îl detectează sunt șanse foarte mari ca următorul să reușească [SH09].

2.2.3 Segmentarea rețelei și principiul apărării stratificate

Configurarea unui firewall sau a unui IPS nu garantează în totalitatea securitatea unei infrastructuri. Dacă toată rețeaua funcționează ca un spațiu unic, fără delimitări interne, un singur dispozitiv compromis poate infecta întreaga infrastructură. Această problemă este rezolvată prin abordarea a două principii arhitecturale: segmentarea rețelei în zone cu niveluri de încredere distincte și aplicarea mai multor straturi independente de protecție.

Modelul bazat pe zone. Un firewall asociază regulile direct cu interfețele fizice de rețea, iar pe măsură ce numărul interfețelor crește, asocierea se face tot mai greu. Modelul *zone-based* rezolvă această problemă prin abstractizare: interfețele sunt grupate în zone logice, iar regulile se asociază zonelor.

O topologie standard include cel puțin trei zone. *Local Area Network* (LAN) este cea mai sigură zonă (*trusted network*), grupează stațiile de lucru și bazele de date, beneficiind de cele mai restrictive politici de intrare. *Wide Area Network* (WAN) reprezintă rețeaua externă, nesigură (*untrusted network*) fiind locul cu cel mai mare volum de trafic și de unde este cel mai posibil să înceapă un atac. *DeMilitarized Zone* (DMZ) funcționează ca o zonă izolată, destinată serverelor cu expunere publică: servere web, servere de e-mail sau servicii DNS [SH09]. Regula esențială a acestei arhitecturi este că traficul inițiat din DMZ spre LAN este blocat implicit. Dacă un server public din DMZ este compromis, atacatorul nu obține acces direct la resursele critice din rețeaua internă. Figura 2.6 ilustrează această topologie de segmentare a zonelor.

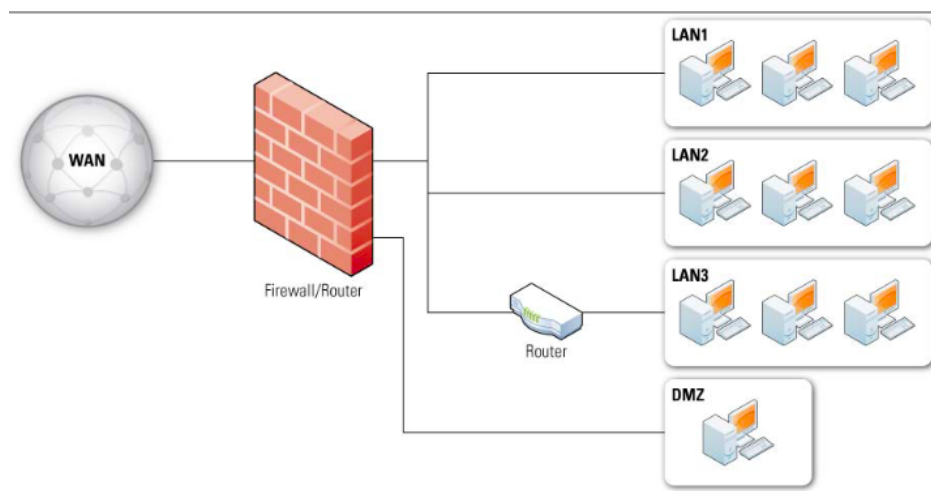


Figura 2.6: Topologie de segmentare LAN, WAN și DMZ [SH09]

Zone suplimentare. Un exemplu este rețeaua pentru oaspeți (*Guest Network*): o zonă separată din rețeaua locală, diferită de DMZ, cu acces la internet dar fără vizibilitate asupra resurselor interne. Aceasta este folosită în instituții de învățământ, hoteluri sau birouri pentru a oferi conectivitate vizitatorilor fără a expune infrastructura organizației.

2.2.4 Firewall-uri de generație următoare (NGFW)

Un firewall tradițional ia decizii pe baza metadatelor sau conexiunilor, dar nu are vizibilitate asupra conținutului efectiv al pachetelor. Un firewall de generație următoare (*Next-generation firewall NGFW*) reprezintă un sistem de securitate avansat, capabil să opereze la toate nivelurile. Acesta oferă funcționalități precum: inspecția profundă a pachetelor, conștientizarea și controlul granular la nivel de aplicație, decriptarea și inspecția traficului securizat (SSL/TLS), maparea politicilor de securitate pe baza identității utilizatorilor.

Inspecția profundă a pachetelor (*Deep Packet Inspection*). Un motor DPI decapsulează pachetul până la nivelul aplicație și analizează payload-ul pentru a găsi semnături sau reguli malițioase [TAD08]. Prin această metodă se pot detecta atacuri de tip SQL Injection, Cross-Site Scripting (XSS) sau *path traversal*, invizibile pentru un filtru care analizează doar antetul.

Conștientizarea la nivel de aplicație (*Application Awareness*). Un NGFW identifică aplicația care generează trafic, indiferent de portul sau protocolul folosit și permite adăugarea de politici de securitate speciale. De exemplu poate permite vizualizarea de fotografii și poate bloca posibilitatea de a le descărca.

Decriptarea și inspecția traficului TLS (*SSL Inspection*). Un motor DPI standard poate analiza conținutul traficului criptat. În schimb un NGFW este capabil să ter-

mine conexiunea TLS, să inspecteze payload-ul decriptat și să reencrpiteze traficul înainte să îl trimită mai departe.

Politici bazate pe identitatea utilizatorului. Dacă într-o organizație zece utilizatori folosesc același IP public, nu există diferențiere per-utilizator. Un NGFW construiește politici de securitate bazate pe grupuri sau utilizatori în locul adreselor IP, folosind un proces în trei pași: extragerea identității (prin monitorizarea activității utilizatorului *Microsoft Active Directory*), mapare IP-to-user (după identificare se creează o asocieră între adresa IP privată, numele utilizatorului și eventual al grupului din care face parte) și aplicarea politicii de securitate specifice.

Capitolul 3

Interceptarea și Procesarea Pachetelor

3.1 Istoricul sistemelor de filtrare în Linux

Sistemul de operare Unix a fost fondat în anul 1969 de către Ken Thompson, Dennis Ritchie, Brian Kernighan, Douglas McIlroy și Joe Ossanna cu scopul de a crea un sistem multi-tasking, multi-user, care a pus bazele sistemelor Linux și macOS. A fost rescris în limbajul C în anul 1973, devenind primul sistem de operare portabil din istorie [TW10].

La Universitatea din Helsinki, în anul 1991, un student pe nume Linus Torvalds a anunțat că lucrează la un sistem de operare pentru calculatoarele x86, un proiect pe care l-a denumit Linux. Inspirația a fost *Minix*, un sistem de tip Unix cu scop educațional creat de Andrew Tanenbaum în 1987 [TW10].

Pe măsură ce Linux și internetul s-au dezvoltat, au fost necesare utilitare de filtrare a pachetelor pentru monitorizarea fluxului de date din rețea.

ipfwadm (IP Firewall Administration) Este primul utilitar în linia de comandă dedicat administrării filtrelor de pachete în Linux, integrat în versiunea 2.0 de kernel, în anul 1996. Permitea definirea de reguli pentru trafic inbound, outbound și forwarded, suportând protocoalele TCP, UDP și ICMP, dar cu filtrare stateless. [Theb]

ipchains A fost introdus în versiunea 2.2 de kernel în anul 1999, a introdus conceptul de lanțuri (*chains*) care a îmbunătățit modul de evaluare și structurarea logică de filtrare a pachetelor [Rus00]. Lanțurile au fost organizate pe tipul de flux al pachetelor INPUT, OUTPUT, FORWARD, iar fiecare lanț putea avea o politică de decizie proprie.

iptables A reprezentat utilitarul de bază pentru filtrarea pachetelor în sistemele Linux timp de două decenii, fiind implementat în versiunea de kernel 2.4 în anul 2001, în interiorul framework-ului Netfilter. A introdus noțiunea de stateful, cu tabela de stări, prin modulul *nf_conntrack*, unde se ține evidența conexiunilor: NEW (pentru primele pachete a unei conexiuni), ESTABLISHED (pentru pachete care

aparțin unei conexiuni deja stabilite), RELATED (pachete care stabilesc o conexiune secundară legată de una activă), INVALID (pentru pachete care nu se potrivesc niciunei conexiuni cunoscute) unde se putea lua decizii bazate pe contextul sesiunii, și tabele cu responsabilități diferite *filter*, *nat*, *mangle*, cu scop de filtrare, traducere de adrese, respectiv modificare a pachetelor. [Sue15]

Tabela 3.1: Evoluția mecanismelor de filtrare a pachetelor în kernelul Linux [Ras07, Sue15, Rus00]

Mecanism	Kernel	Contribuție principală	Limitare
<i>ipfwadm</i>	v2.0 (1996)	Prima interfață CLI, suport reguli inbound/outbound/forward	Fără <i>connection tracking</i> ; structuri neextensibile
<i>ipchains</i>	v2.2 (1999)	Chains; politici implicite per lanț	Fără <i>connection tracking</i>
<i>iptables</i>	v2.4 (2001)	5 hook-uri Netfilter; <i>conn-track table</i>	Evaluare O(N); sintaxe separate IPv4/IPv6
<i>nftables</i>	v3.13 (2014)	Bytecode VM; seturi O(1); sintaxă unificată <i>inet</i>	

3.2 Subsistemul Netfilter

Proiectul Netfilter a fost fondat de Paul „Rusty” Russell în anul 1999, cu scopul de a îmbunătății sistemele *ipchains* și *ipfwadm*, dând naștere la modulul *iptables*. Este un cadru arhitectural integrat direct în stiva de rețea a nucleului Linux, care permite interceptarea, modificarea și filtrarea pachetelor [Sue15]. Rolul său este de a expune puncte de interceptare fixe, 5 momente cheie denumite *hooks* pe traseul logic pe care îl parcurge un pachet de la intrarea pe interfața fizică, până la aplicația destinație, și invers. Aceste puncte sunt: PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING, fiecare dintre acestea permit reguli speciale la fiecare punct. Figura 3.1 realizează o imagine de ansamblu asupra drumului parcurs de pachetele interceptate prin aceste puncte.

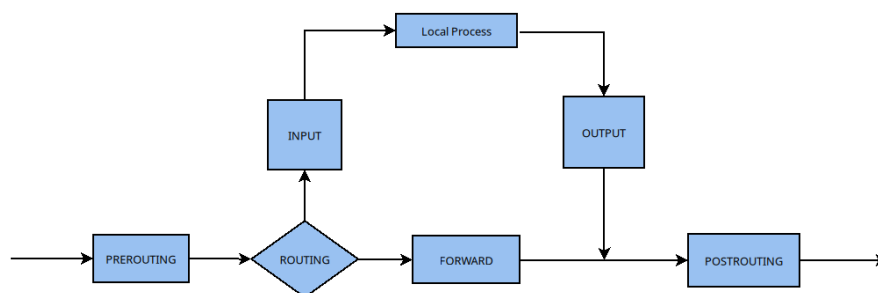


Figura 3.1: Reprezentarea momentelor cheie Netfilter

PREROUTING. Este primul punct de interceptare, atins de orice pachet când ajunge pe interfața de rețea, înainte de orice decizie de rutare. Aici se aplică de obicei regulile de DNAT și se inițializează tabela de stări pentru pachetele noi;

INPUT. De acest hook se agață pachetele care au fost rutate către mașina locală (*localhost*), unde traficul este filtrat pentru servicii sau aplicații locale;

FORWARD. Atins exclusiv de pachetele care tranzitează mașina, adică pachetele care au ajuns pe o interfață și sunt rutate spre o altă interfață, pachetele care nu sunt destinate unui proces local;

OUTPUT. Acest hook este destinat pachetelor generate de procesele locale ale mașinii înainte ca acestea să ajungă înapoi pe interfața de rețea;

POSTROUTING. Este ultimul punct de interceptare unde ajung pachetele după ce au fost luate deciziile de rutare și sunt pregătite să părăsească interfața prin placa de rețea. Aici se aplică regulile pentru SNAT.

3.2.1 Arhitectura nftables

nftables este utilizarea modernă de filtrare a pachetelor în Linux, disponibilă cu versiunea kernel 3.13 (2014), care înlocuiește *iptables*, *ip6tables* și *arptables* printr-o singură interfață unificată. Reutilizează infrastructura hook-urilor, tabela de stări, NAT și mecanismul de *userspace queue* și *logging*, oferind un mod de execuție a regulilor mult mai performant față de predecesorii săi. [Theb]

Elementul central al arhitecturii *nftables* este o mașină virtuală (*Virtual Machine VM*) specializată pentru procesarea pachetelor, integrată direct în kernel [Theb]. Când un administrator definește un set de reguli folosind instrumentul din linia de comandă *nft*, acesta nu le trimite direct în kernel ca text sau structuri de date fixe, se compilează întregul set de reguli în *bytecode* și îl transmite prin interfața *Netlink*. Kernelul stochează *bytecode*-ul și îl execută prin intermediul VM-ului la fiecare pachet interceptat de un hook. Procesul este reversibil: la interogarea setului de reguli curent, VM-ul decompilează *bytecode*-ul înapoi în reprezentarea text originală. [Sue15, Theb]. Figura 3.2 reprezintă ierarhia prin care *nftables* organizează modul de adresare a politicilor de filtrare.

O tabelă (*table*) este un container logic care grupează lanțuri asociate unui protocol sau unei funcționalități. Spre deosebire de *iptables*, care venea cu cinci tabele predefinite, *nftables* nu include nicio tabelă implicită, totul este definit de administrator [Sue15]. Resursele sunt alocate exclusiv pentru o anumită configurare. De exemplu un firewall pe un host care nu face rutare nu va înregistra niciodată un *chain* de FORWARD activ.

Un lanț (*chain*) este un container de reguli atașat unui hook. Fiecare lanț are asociat un tip care îi definește scopul: filter pentru filtrarea pachetelor, nat pentru

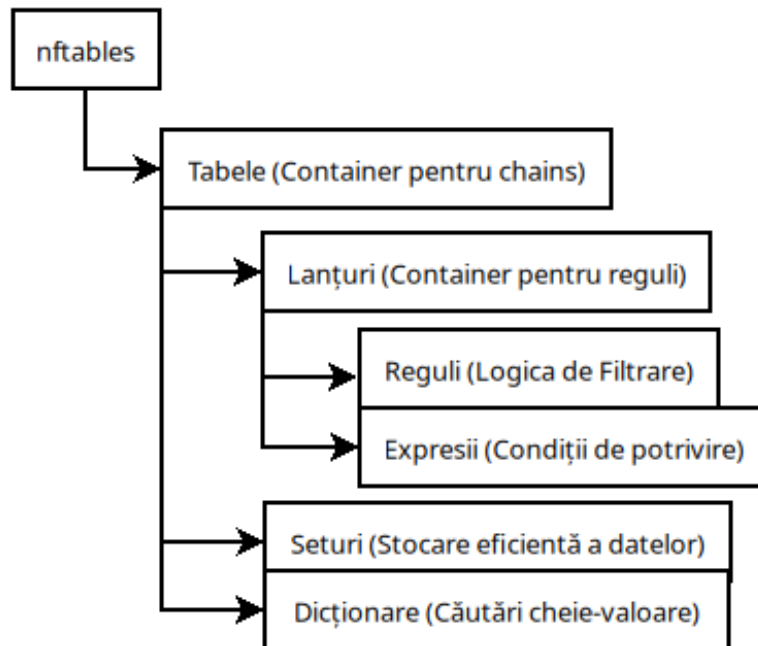


Figura 3.2: Ierarhia sistemului nftables

translatarea adreselor de rețea și route pentru reevaluarea rutei după modificarea unui câmp din antetul IP [Sue15].

Setul de reguli este locul în care are loc filtrarea pe baza logicii definite de administrator, fiind o potrivire între o regulă și un verdict. Comenzile pentru manipula-rea regulilor sunt: add (se adaugă o regulă la finalul lanțului), insert (se inserează o regulă la începutul lanțului sau la o locație specificată) și delete (se șterge o regulă din lanț) Verdicturile disponibile în nftables sunt [Sue15]:

- accept - pachetul este acceptat, procesarea în lanțul curent se oprește și pachetul continuă spre destinație;
- continue - se continuă procesarea pachetului;
- drop - pachetul este eliminat silențios;
- jump - procesarea este transferată unui alt lanț, cu revenire în cel apelant după finalizare;
- goto - similar cu jump, fără revenire;
- limit - procesarea pachetului se face conform regulii doar dacă a nu a fost atinsă limita de pachete de acel tip;
- queue - procesarea pachetului este oprită iar pachetul este redirectionat către alt proces de analiză;
- return - oprește procesarea în chain-ul curent și revine în cel apelant;

- log - înregistrează pachetul în jurnalul sistemului și continuă procesarea cu regula următoare.

Una dintre îmbunătățirile de performanță ale nftables față de generațiile anterioare o reprezintă implementarea nativă de seturi (*sets*) și dicționare (*maps*). Un set este o colecție de elemente (adrese IP, porturi, protocoale sau combinații ale acestora), iar un dicționar permite mapări predefinite (ex: port - ip: type inet.service: ipv4_addr), ambele stocate în structuri de date optimizate pentru căutare rapidă (*hash tables* $O(1)$ sau *binary tree* $O(\log n)$) [Theb]

Un alt avantaj important este înlocuirea atomică (*atomic rule replacement*) a setului de reguli. În *iptables* modificarea unui chain presupunea extragerea întregului set de reguli din nucleu în memorie, actualizarea și reîncărcarea acestuia, ceea ce genera o fereastră de timp în care traficul era procesat pe un set de reguli inconsistent sau era blocat total. Pentru nftables, actualizarea setului de reguli se realizează simplu, printr-un singur apel de sistem.

3.2.2 Comunicarea kernel-userspace

Netlink este mecanismul standard din kernelul Linux pentru comunicarea între procesele din spațiul utilizator și componentele kernelului, înlocuind interfața rigidă *ioctl()* cu un protocol de mesagerie asincron, extensibil și bidirecțional. Comunicarea se realizează printr-un socket din familia AF_NETLINK, folosind canalul NETLINK_NETFILTER (canal dedicat întregului ecosistem Netfilter) [Theb].

Mesajele schimbate prin *nfnetlink* folosesc un format extensibil de tip TLV (*Type-Length-Value*), în care fiecare atribut al unui mesaj este identificat printr-un tip, urmat de lungimea sa și de valoarea efectivă. Sunt expuse trei subsisteme principale, fiecare responsabile pentru un tip distinct de comunicare cu Netfilter. Subsistemul *nf_tables* transportă operațiile de administrare a regulilor nftables și răspunsurile kernelului la interogări, inclusiv valorile curente ale contoarelor asociate regulilor. Subsistemul *nf_conntrack_netlink* expune tabela de stări ale conexiunilor active. Subsistemul *nfnetlink_queue* este responsabil pentru livrarea pachetelor din kernel către spațiul utilizator.

NFQUEUE (*Netfilter Queue*) este mecanismul prin care un pachet interceptat de o regulă nftables cu verdictul *queue num N* este transferat din kernel către un proces userspace pentru analiză și decizie [Thea]. Când un pachet atinge o astfel de regulă, kernelul oprește procesarea sa în stiva Netfilter și serializează metadatele acestuia împreună cu payload-ul complet, de la Layer 2 până la Layer 7, într-un mesaj *nfnetlink_queue*. Mesajul este pus într-o coadă internă a kernelului identificată prin numărul *N*, unde rămâne suspendat până la primirea unui verdict. Procesul userspace este abonat la coadă prin biblioteca *libnetfilter_queue*. Citește pachetul

prin același socket Netlink, îl inspectează după propria logică și emite un verdict (NF_ACCEPT, NF_DROP, NF_QUEUE, NF_STOLEN) explicit înapoi spre kernel. La primirea verdictului, kernelul reia procesarea pachetului: îl eliberează în stivă, îl elimină sau îl retrimite în capul cozii pentru reevaluare. Această arhitectură pune toată logica de decizie în userspace, unde poate fi implementată în orice mod se dorește după nevoie, în orice limbaj de programare, fără module kernel suplimentare [PyP23].

3.3 Soluții alternative

Deși infrastructura Netfilter și sistemul de operare Linux domină piața echipamentelor de rețea și a mediilor cloud, o perspectivă arhitecturală completă asupra tehnologiilor de filtrare necesită menționarea soluțiilor implementate și pe alte platforme. Sistemele de operare Windows și macOS au dezvoltat mecanisme interne distincte pentru a asigura inspecția și controlul traficului la nivelul stațiilor de lucru.

3.3.1 Windows Filtering Platform (WFP)

Windows Filtering Platform (WFP) este un set de API-uri și servicii de sistem care oferă o platformă pentru crearea de aplicații de filtrare a traficului de rețea integrat în nucleul Windows, disponibil începând cu Windows Vista și Windows Server 2008 [Mic24b]. WFP nu expune un set de reguli predefinite, ci oferă o interfață unificată prin care permite componentelor native ale sistemului de operare să lucreze împreună cu aplicațiile de securitate dezvoltate de producători, fără să implementeze propria logică de interceptare la nivelul nucleului.

Arhitectura WFP este structurată pe două planuri de execuție distincte, prezentate în Figura 3.3, care cuprinde un plan la nivelul nucleului (*Kernel Mode*), unde se execută clasificarea efectivă a pachetelor, și un plan la nivelul utilizator (*User Mode*), unde se administrează politicile și se gestionează logica de filtrare a pachetelor.

Motorul de bază de filtrare (*Base Filtering Engine bfe*) este componenta centrală a arhitecturii WFP și rulează în spațiul utilizator, ca serviciu al sistemului de operare [Mic24b]. BFE coordonează toate celelalte componente ale platformei: stochează politicile de filtrare configurate de aplicații sau de administrator, gestionează ciclul de viață al filtrelor (adăugare, modificare, ștergere) și le propagă spre motorul de filtrare din nucleu (*KM Filter Engine*); comunicarea dintre BFE și nucleu se realizează prin interfața *IOCTL*. Toate aplicațiile din spațiul utilizator care doresc să înregistreze filtre interacționează cu BFE exclusiv prin biblioteca de management *fwpuclnt.dll*. Această bibliotecă este partajată de toate componentele din zona User Mode: aplicațiile socket, firewall-urile third party, Windows Firewall și serviciul de

Windows Filtering Platform Architecture Overview

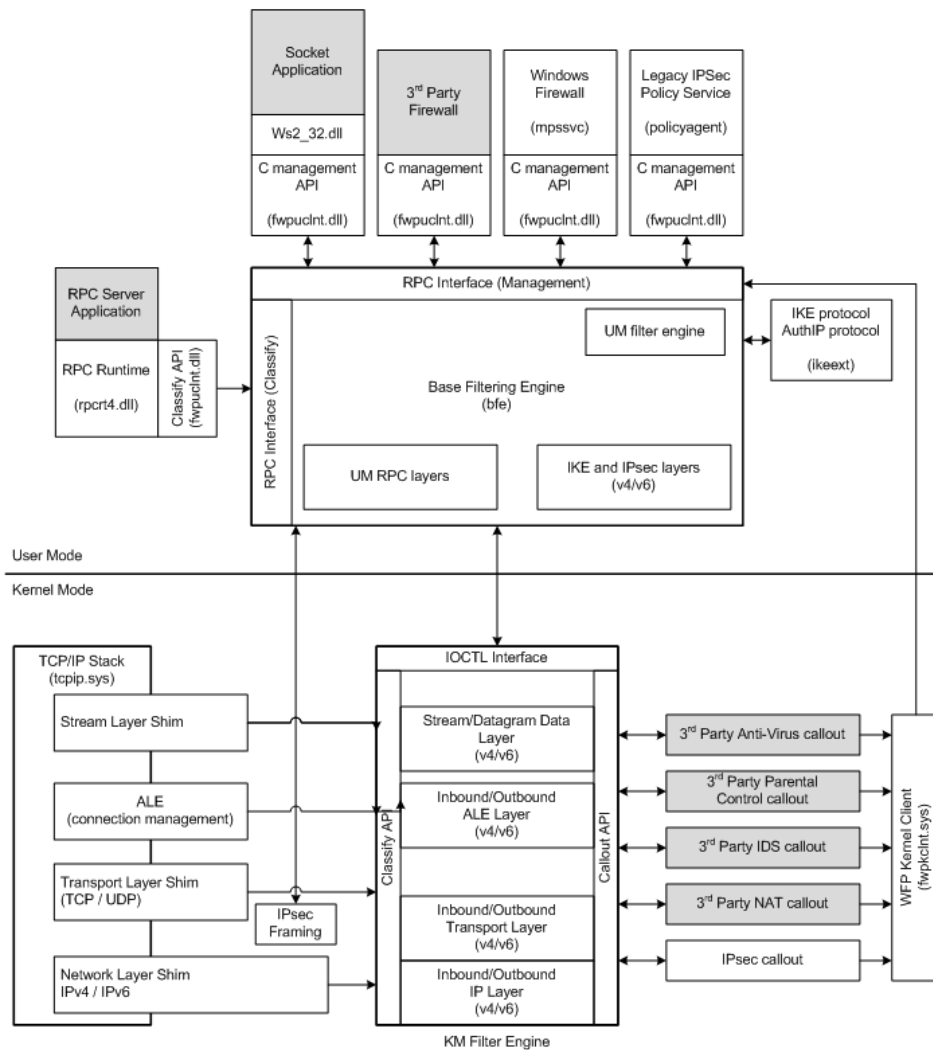


Figura 3.3: Arhitectura Windows Filtering Platform [Mic24b]

politici IPsec moștenite (*Legacy IPsec Policy Service*).

Straturile de filtrare sunt punctele predefinite din stiva de rețea a nucleului unde pachetele pot fi oprite și evaluate [Mic24b]. Fiecare corespunde unui moment specific al pachetului și poate impune condiții sau acțiuni proprii. Acestea sunt organizate pe nivelurile stivei, reprezentate ca blocuri distincte în zona KM Filter Engine.

Mecanismul de clasificare este procesul prin care motorul de filtrare Kernel determină acțiunea aplicată unui pachet la fiecare strat [Mic24b]. La sosirea unui pachet pe un nivel, motorul parcurge lista filtrelor, iar la prima potrivire cu metadatele pachetului se determină acțiunea: *permit* eliberează pachetul spre nivelul următor al stivei, *block* îl elimină, iar *callout* îl redirecționează către un modul specializat de inspecție aprofundată. Dacă niciun filtru nu se potrivește, se aplică *default policy*.

Un driver callout extinde capabilitățile Windows Filtering Platform prin filtrarea

pachetelor în moduri mai avansate decât simpla decizie bazată pe antele pachetelor [Mic24a]. Acesta rulează în spațiul nucleu, se înregistrează la KM Filter Engine și permite funcționalități precum: inspecție aprofundată a pachetelor, modificarea pachetului, modificarea fluxului, sau înregistrarea datelor (se poate număra câte pachete au fost respinse pe baza unui filtru anume). În plus, driverele callout pot adăuga la rândul lor filtre adiționale în Base Filtering Engine. [Mic24b]

Windows Defender Firewall este sistemul default de securitate a sistemului de operare Windows, construit în întregime peste WFP. Regulile de firewall configurate de către administrator prin interfața grafică sau prin *Group Policy* sunt traduse intern în filtre WFP și înregistrate la straturile corespunzătoare din motorul de filtrare.

3.3.2 Packet Filter (PF) pentru macOS

Packet Filter (PF) este un utilitar creat în anul 2001, ca răspuns la o incompatibilitate de licență pentru *IPFilter*, cu funcționalități pentru filtrarea pachetelor și modelarea traficului. A fost dezvoltat inițial pentru sistemul de operare OpenBSD (sistem de operare open-source bazat pe BSD, creat în anul 1995) [Han14]. PF este integrat direct în stiva de rețea a nucleului și procesează fiecare pachet printr-o succesiune fixă de etape: normalizarea traficului, verificarea tabelii de stări, evaluarea setului de reguli și aplicarea verdictului final.

Normalizarea traficului (*scrubbing*) constituie prima etapă din pipeline-ul PF și operează exclusiv asupra structurii pachetelor, înainte ca acestea să fie supuse oricărei decizii de filtrare [Han14]. Scopul acestei etape este de a aduce fiecare pachet într-o formă completă și neambiguă. Operația centrală a normalizării este reasamblarea fragmentelor (*fragment reassembly*). Când un pachet IP depășește unitatea maximă de transmisie a unui segment de rețea, routerile intermediare îl fragmentează în bucăți mai mici, fiecare cu propriul antet IP. Packet Filter interceptează fragmentele, le acumulează și le reassemblează într-un pachet complet înainte de evaluare. Fragmentele invalide sunt eliminate, întrucât pot fi exploatate pentru a ocoli reguli de filtrare și pot introduce pachete dăunătoare. O altă operație disponibilă este plafonarea valorii *Maximum Segment Size*, care previne situațiile în care pachetele prea mari nu pot traversa anumite segmente de rețea. Normalizarea include și mecanismul *antispoof*, care respinge pachetele cu adrese sursă ce nu pot fi rutate legitim prin interfața de pe care au sosit, oferind protecție împotriva atacurilor de falsificare a adresei IP.

Tabela de stări (*state table*) oferă mecanismul de inspecție stateful. Pachetele asociate unei sesiuni existente nu mai necesită parcurgerea setului de reguli, ceea ce duce la îmbunătățirea performanței. Fiecare intrare din tabela de stări are asociat un mecanism de expirare bazat pe intervale de timp specifice protocolului.

Evaluarea setului de reguli este etapa în care PF decide verdictul fiecărui pachet care nu a putut fi asociat cu o intrare existentă din tabela de stări. Setul de reguli are două componente structurale care îi cresc flexibilitatea: tabelele de adrese și ancorele. Tabelele de adrese (*address tables*) sunt structuri de date care rețin colecții de adrese IP sau prefixe de rețea și sunt optimizate pentru căutare rapidă. Ancorele (*anchors*) sunt sub-seturi de reguli, definite în setul principal ca puncte de extensie [Han14]. Când PF evaluează o regulă care referă o ancoră, execuția se ramifică temporar, iar dacă nicio regulă din ancoră nu produce un verdict final, evaluarea continuă în setul principal. Logica de evaluare a regulilor în PF urmează principiul că ultima regulă care se potrivește determină verdictul. Regulile sunt parcurse secvențial, iar fiecare potrivire suprascrie decizia regulii anterioare. Excepția de la această logică este cuvântul cheie *quick*: când un pachet se potrivește cu o regulă marcată astfel, evaluarea setului de reguli se oprește imediat, iar decizia acelei reguli este aplicată fără a verifica cele rămase.

Verdictul reprezintă decizia finală aplicată pachetului după parcurgerea tuturor etapelor. PF oferă două acțiuni principale, *pass* și *block*. Acțiunea *pass* transmite pachetul mai departe în stiva de rețea, iar dacă regula include directiva de menținere a stării, se creează simultan o intrare în tabela de stări pentru sesiunea respectivă. Acțiunea *block* oprește pachetul și se manifestă în două variante: *drop* (elimină pachetul silențios) și *return* (elimină pachetul și notifică).

Capitolul 4

Dezvoltarea Sistemului CuciSec

4.1 Arhitectura sistemului

Sistemul CuciSec este structurat în două zone arhitecturale cu responsabilități diferite: kernel-space și userspace. Această decizie de design pornește de la ideea că anumite operații necesită eficiență ridicată, iar altele necesită analiză amănunțită. Filtrarea bazată pe adrese IP sau pe zone, aplicarea limitelor de adrese pe secundă și menținerea stărilor conexiunilor sunt operații repetitive, cu latență critică, care beneficiază de execuția directă în nucleul sistemului de operare, fără overhead de copiere a datelor între spații de memorie. Logica de inspecție avansată, evaluarea regulilor și detecția amenințărilor din payload-ul pachetelor necesită o abordare mai lentă, dar necesară.

Stratul kernel-space este implementat folosind utilitarul *nftables*, organizând regulile într-un pipeline de filtrare stratificat care analizează seturi pentru adrese binate, adrese provenite din zone sigure, reguli pentru blocarea atacurilor volumetrice, porturi capcană și creează legătura cu userspace pentru analiza detaliată. Stratul este conceput pentru performanța sistemului, prevenirea suprasolicitării cu pachete repetitive și gestionarea conexiunilor.

Stratul userspace primește pachetele prin biblioteca *NetfilterQueue* (wrapper Python peste biblioteca *libnetfilter_queue* scrisă în limbajul de programare C) și le procesează printr-un pipeline decizional secvențial. Un server *FastAPI*, pornit la inițializarea sistemului alături de celelalte componente, expune interfața REST prin care o aplicație web administrează regulile, statisticile, jurnalizarea și adresele blocate.

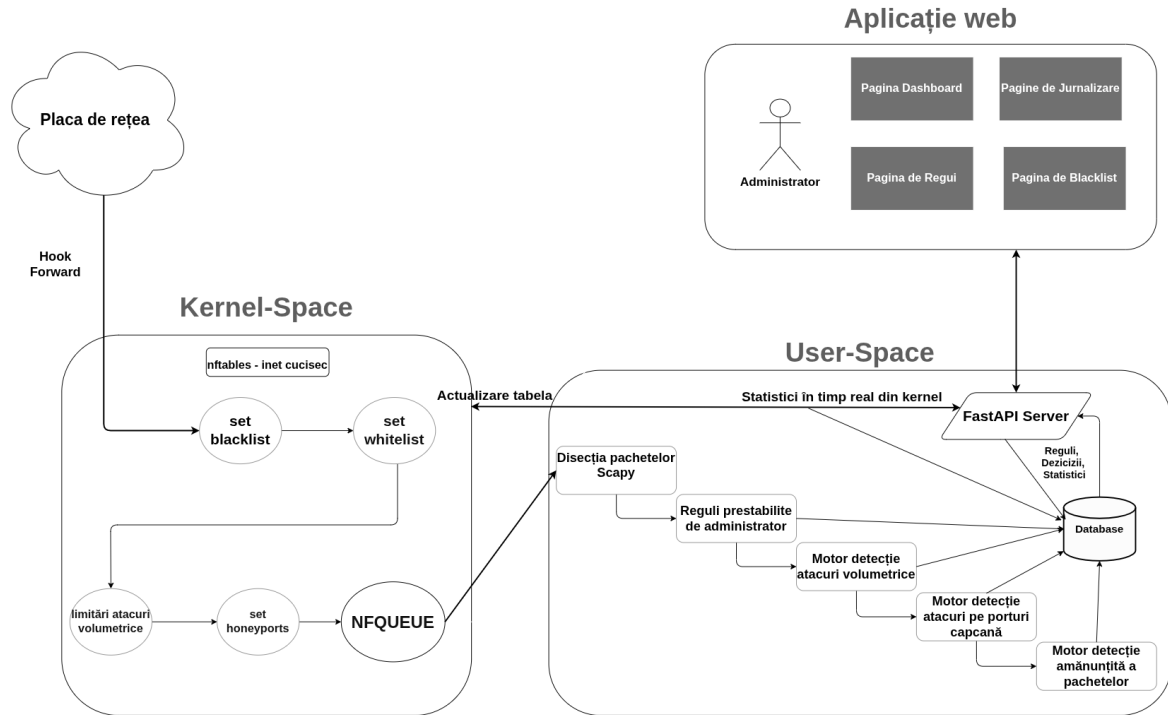


Figura 4.1: Arhitectura sistemului CuciSec

4.2 Stratul kernel-Space

Stratul kernel asigură performanța sistemului prin utilitarul *nftables*, cu setul de reguli organizat în tabela `inet cucisec`. Familia `inet` unifică regulile pentru protocoalele IPv4 și IPv6 într-o singură tabelă, eliminând discrepanța *iptables* de a menține seturi dublate de reguli pentru adrese IP cu versiuni diferite. Tabela este definită pe lanțul forward, interceptând pachetele care tranzitează sistemul și aplicând regulile de filtrare stabilite.

Lanțul FORWARD aplică regulile în nouă etape, verdictele rapide și fără overhead computațional sunt plasate primele, astfel încât traficul clar malitios sau clar legitim este filtrat înainte de a consuma resurse suplimentare. Prima etapă elimină pachetele cu stare de conexiune `invalid`, o categorie care include pachete corupte (payload sau headere alterate) sau pachete care încalcă protocoalele de la nivelul transport. A doua etapă consultă setul `blacklist`: orice pachet provenit dintr-o adresă interzisă este eliminat imediat. A treia etapă acceptă direct pachetele din setul `whitelist`, fără să parcurgă restul regulilor; această poziție garantează că traficul LAN legitim nu este întârziat sau redirectionat spre userspace. Etapele patru și cinci implementează detecția inundațiilor volumetrice prin limite de rată globale și per-destinație. Limitele globale sunt implementate pentru atacurile de tip

DDoS, în care nicio sursă individuală nu depășește un prag per-IP și volumul agregat nu depășește capacitatea de procesare. Limitele per-destinație adaugă protecție împotriva atacurilor concentrate pe o singură țintă, indiferent de sursa lor. Etapa șase direcționează către userspace pachetele destinate porturilor HTTP și server, singurele pentru care motorul de inspecție avansată poate analiza payload-ul. Etapa șapte aplică limite de rată per-sursă, un nivel de protecție complementar față de limitele globale din etapele anterioare, fiind vorba doar de atac Denial of Service de la o singură sursă. Astfel, prin această regulă administratorul poate urmări cu atenție dacă este trafic de la o sursă legitimă, sau este un atac, având o fereastră de câteva secunde să ia o decizie; în cazul în care traficul atinge limita pe toată această durată, este considerat malițios și este eliminat. Etapa opt direcționează spre userspace orice conexiune pe un port din setul `honey_ports`. Etapa nouă este o regulă *catch-all* care trimite în userspace tot traficul rămas fără verdict explicit; niciun pachet care a trecut de filtrele kernelului nu traversează sistemul fără să fie inspectat de logica din userspace, având asigurat protecția împotriva unui *bottleneck* din regulile anterioare.

Fiecare regulă semnificativă are asociat un contor explicit: `blacklist_drop`, `global_syn_flood`, `dst_syn_flood`, `tcp_syn_flood`, `udp_flood`, `icmp_flood` și `honeyport_hits`. Un thread dedicat interoghează periodic acești contori prin comanda `nft -j list ruleset`, care returnează valorile curente în format JSON și calculează diferențe cumulative stocate în baza de date. Statisticile din dashboard reflectă astfel valorile acumulate pe întreaga durată de funcționare a sistemului, nu doar de la ultimul restart. La oprirea sistemului, tabela `cucisec` este ștearsă kernel, restaurând comportamentul implicit al traficului.

4.3 Stratul userspace

Pachetele redirectionate fără un verdict explicit ajung în userspace prin mecanismul NFQUEUE. Componenta PacketInterceptor se leagă la coada cu numărul 1 prin biblioteca `NetfilterQueue` și apelează o funcție *callback* invocată sincron pentru fiecare pachet primit. Thread-ul principal este atașat acestei funcții pe toată durata de viață a sistemului; kernelul suspendă fiecare pachet în buffer până la primirea verdictului. Fiecare pachet trebuie să primească un verdict înainte de a prelua următorul din coadă.

Primul pas este disecția pachetului de către clasa `PacketAnalyzer`. Biblioteca `Scapy` reconstruiește structura completă a antetelor și extrage câmpurile relevante: adresele sursă și destinație, protocolul, porturile și payload-ul. Rezultatul disecției este un obiect `PacketInfo`, care encapsulează toate câmpurile extrase și este transmis tuturor etapelor decizionale.

RuleEngine este prima etapă de decizie și are prioritatea maximă în pipeline: un

verdict emis de aceasta încheie procesarea imediat, fără ca celelalte motoare să mai fie consultate. La pornirea sistemului regulile sunt încărcate din baza de date într-o listă în memoria RAM, pentru a evita orice fel de latență care poate fi critică. Fiecare regulă specifică zona de rețea căreia i se aplică: LAN, WAN sau ANY; clasificarea sursei se face prin compararea adresei IP a pachetului cu o listă de subrețele private predefinite. Potrivirea adreselor suportă notație CIDR (192.168.1.0/24), adresă exactă, sau wildcard (*).

FloodEngine detectează atacurile volumetrice per-sursă într-un anumit interval de timp. La fiecare pachet, fereastra se recalculează prin eliminarea *timestamp-urilor* mai vechi de `TIME_WINDOW = 12` secunde, adăugarea timestamp-ului curent și numărarea elementelor rămase. Dacă numărul depășește pragul configurat per protocol pachetul este eliminat și sursa este interzisă. Cheia compusă (`ip, protocol`) garantează că pragurile sunt independente: un IP care trimite trafic UDP în volum mare nu consumă din cota TCP a aceleiași surse, evitând blocaje false pentru traficul legitim mixt.

HoneyportEngine implementează conceptul de port capcană: orice tentativă de conexiune pe un port asociat unui serviciu vulnerabil sau nefolosit reprezintă o posibilitate foarte mare de activitate malițioasă, deoarece niciun trafic legitim nu inițiază brut conexiuni pe aceste porturi [Spi02]. Sunt predefinite următoarele porturi: 21 (FTP), 23 (Telnet), 25 (SMTP), 111 (Portmapper/RPC), 445 (SMB), 2323 (Telnet alternativ), 3389 (RDP), 4444 (listener implicit Metasploit) și 9999 (backdoor).

DPIEngine (*Deep Packet Inspection Engine*) inspectează payload-ul și identifică tipare de atac prin potrivire de expresii regulate [TAD08]. Motorul este activ pe porturile HTTP (80, 8080, 8000 și 8443). Expresiile regulate sunt organizate în șapte categorii de atac *SQL injection* (construcții de tip "UNION SELECT", "DROP TABLE", condiții "1 = 1"), *cross-site scripting* (taguri "<script>", atribut "javascript:", "handler-uri" de evenimente), *injectare de comenzi de sistem* (" ; cat", "| whoami", "wget", *path traversal* (secvențe ".../.../", "/etc/passwd", ".../.../etc/hosts"), *Remote Code Execution* (funcții "system()", "exec()", "cmd.exe"), *Server-Side Request Forgery* (SSRF)(metadata cloud "169.254.169.CCC") și *scanere* (User-Agent de tip "sqlmap", "nikto", "masscan", "nmap"). Înainte de potrivire, payload-ul este normalizat, decodează secvențele URL și convertește totul la litere mici. Fără această normalizare, un atacator ar putea ocoli toate semnăturile prin codarea caracterelor relevante: %55NION %53ELECT devine union select după decodare și este detectat de semnătura SQL injection. La primul match, procesarea se oprește și adresa sursa este blocată, fără a continua verificarea celorlalte semnături.

Pachetele care parcurg toate cele patru etape fără să declanșeze un verdict de blocare sunt acceptate implicit și redirectionate spre destinație. Evenimentul este înregistrat în baza de date cu acțiunea `DEFAULT_ACCEPT` furnizând vizibilitate com-

pletă asupra traficului legitim în dashboard-ul de monitorizare.

Indiferent de motorul care a emis verdictul, acțiunea de blocare urmează același flux în componenta FirewallActions. Înainte de orice operație, adresa sursă este verificată în setul in-memory `_banned_ips`; dacă este deja prezentă, procesarea se încheie fără acțiuni suplimentare. Dacă adresa este nouă, este adăugată în set, injectată imediat în setul `blacklist` și stocată în baza de date. Din momentul injectării, pachetele ulterioare din aceeași sursă sunt eliminate direct în kernel beneficiind de paradigma *hot-reload*. La pornirea sistemului, setul `_banned_ips` este preîncărcat din baza de date, astfel încât interdicțiile din sesiunile anterioare rămân active fără nicio intervenție manuală.

4.4 Server și interfața web

Interfața de management a sistemului este expusă printr-un server *FastAPI* care rulează pe portul 8000 ca thread *daemon*, pornit în paralel cu funcția callback pentru NFQUEUE la inițializarea sistemului. Separarea pe alt thread garantează că o eroare în stratul API nu afectează procesarea pachetelor din fluxul principal.

Tabelul 4.1 sintetizează cele opt endpoint-uri REST expuse de server. Endpoint-urile de reguli suportă operațiile CRUD complete și o operație de activare/dezactivare; orice modificare declanșează automat `reload_rules()` în RuleEngine, făcând noua configurație activă imediat. Endpoint-urile de log-uri oferă listare paginată cu filtre pe protocol, acțiune și adresă sursă, plus agregarea evenimentelor pe minut din ultimele 60 de minute, utilizată de graficul de trafic din dashboard. Endpoint-urile de blacklist expun ban și unban manual, care stochează sau elimină adresa IP, simultan, în baza de date, în setul `nftables` și cache-ul in-memory, restricționând sau restaurând accesul fără a necesita restart.

Tabela 4.1: Endpoint-urile REST API ale sistemului CuciSec

Metodă	Rută	Descriere
GET / POST	<code>/api/rules/</code>	Listare și adăugare reguli
PUT / DELETE	<code>/api/rules/{id}</code>	Editare și ștergere regulă
PATCH	<code>/api/rules/{id}/toggle</code>	Activare sau dezactivare regulă
GET	<code>/api/logs/</code>	Listare log-uri cu filtre și paginare
GET	<code>/api/logs/count</code>	Agregare evenimente pe minut (60 min)
GET / POST	<code>/api/blacklist/</code>	Listare și adăugare ban manual
DELETE	<code>/api/blacklist/{ip}</code>	Unban manual DB și <code>nftables</code>
GET	<code>/api/stats/</code>	Statistici agregate din DB și <code>nftables</code>

Endpoint-ul GET `/api/stats/` combină date din două surse. Valorile cumulative: numărul total de log-uri, pachetele acceptate și respinse, IP-urile interzise și ban-urile recente, sunt preluate din baza de date SQLite. Contorii de flood și DPI provin dintr-o combinație între valoarea acumulată în tabela `KernelCounters` din baza de date, actualizată periodic, un snapshot la interval de 10 secunde, și diferența curentă față de ultimul snapshot, calculată din statisticile preluate de la nivel de kernel. Prin această abordare, statisticile afișate reflectă atât activitatea istorică, stocată în baza de date, cât și evenimentele în timp real.

Interfața web este o aplicație *React* construită cu *Vite* și stilizată cu *Tailwind CSS*. Graficele au fost construite folosind biblioteca *recharts*, tabelele, cardurile pentru metrice și formularele au fost implementate cu ajutorul bibliotecii *shadcn/ui*. Dashboard-ul principal, ilustrat în Figura 4.2, centralizează statisticile sistemului în timp real și un grafic de trafic care vizualizează volumul evenimentelor pe minut din ultimele 60 de minute, oferind administratorului o imagine de ansamblu asupra activității curente. Pagina de reguli, ilustrată în Figura 4.3, prezintă lista de reguli configurate cu opțiuni de adăugare, editare, activare/dezactivare și ștergere. Pagina de blacklist, ilustrată în Figura 4.4 afișează adresele banate, alături de motivul și momentul în care au fost blocate. Pagina de loguri, prezentată în Figura 4.5, realizează un istoric informațional asupra fluxului de pachete analizate.

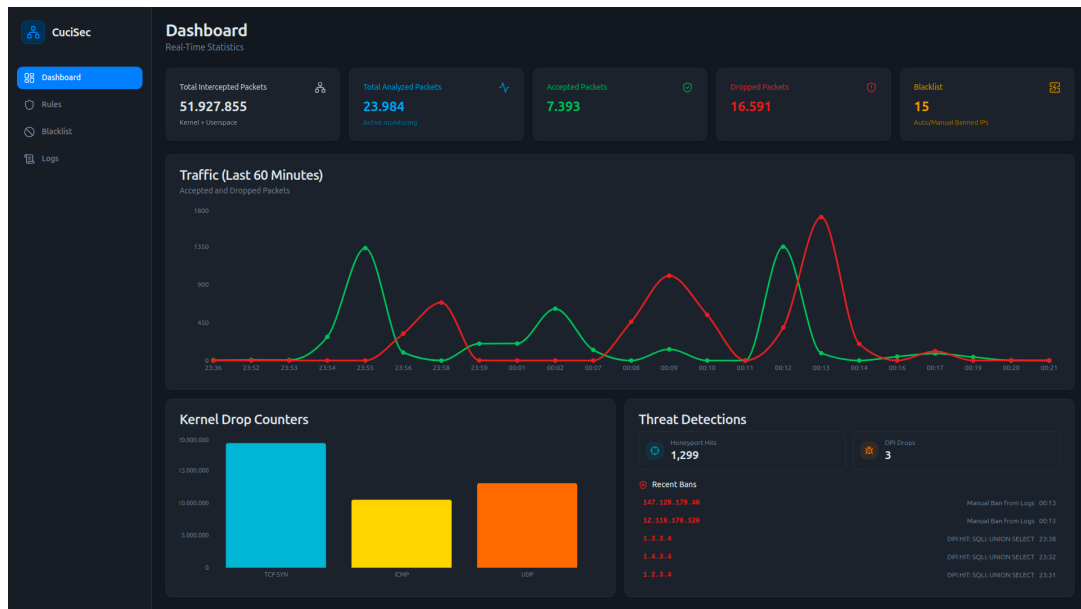


Figura 4.2: Dashboard-ul principal al sistemului Cucisec

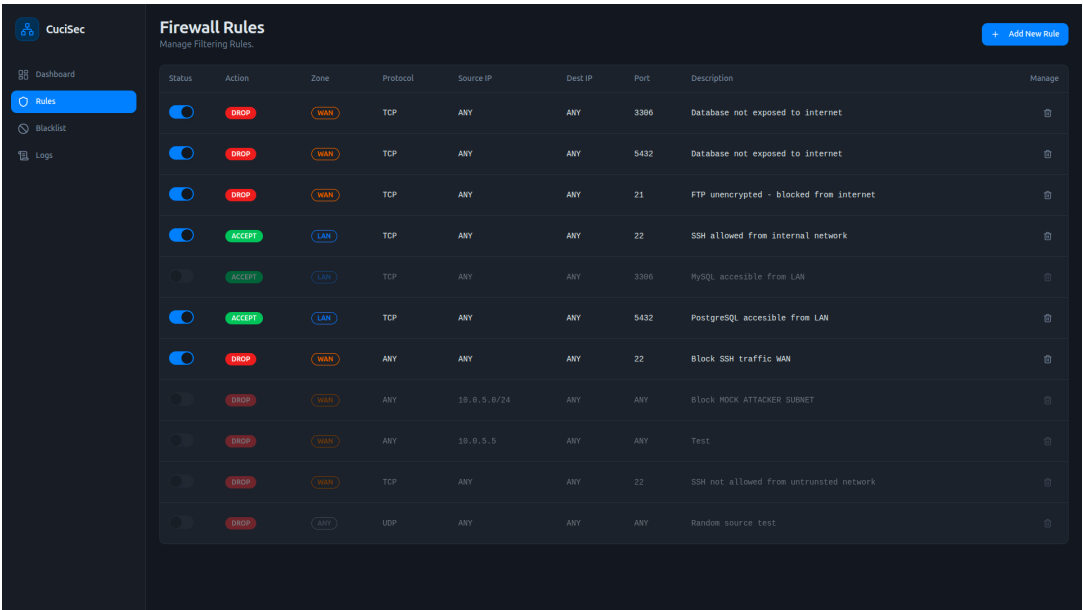


Figura 4.3: Pagina de administrare a regulilor de filtrare

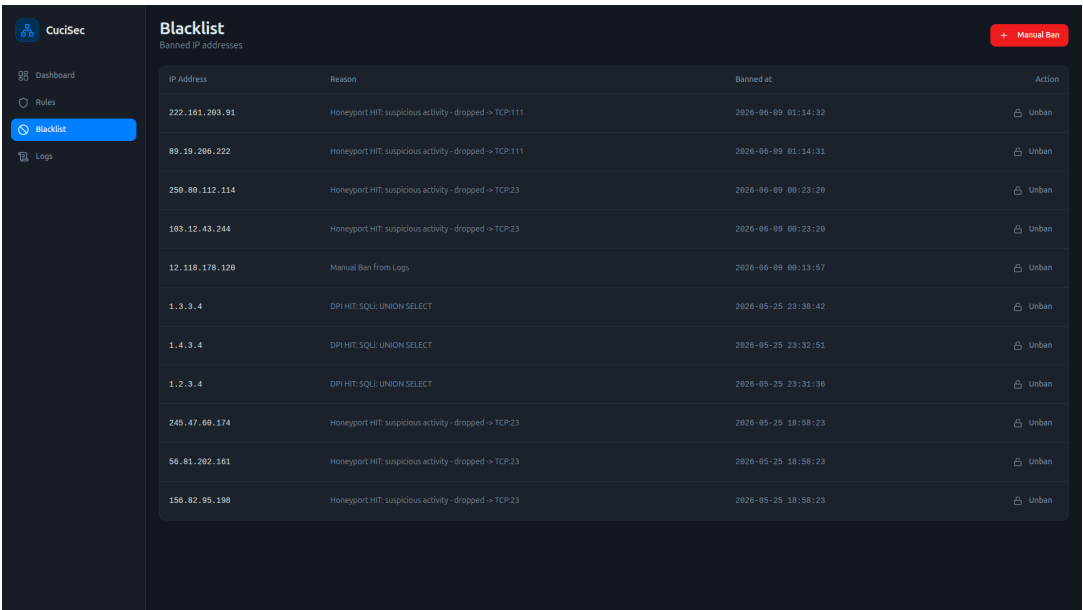


Figura 4.4: Pagina de gestiune a listei negre

Time	Protocol	Source	Destination	Action	Details	Ban
5:37:52 PM	TCP	62.174.86.4:1477	10.0.3.5	ACCEPT	DEFAULT_ACCEPT	
5:37:51 PM	TCP	28.123.169.244:1476	10.0.3.5	ACCEPT	DEFAULT_ACCEPT	
5:37:50 PM	TCP	212.31.53.145:1475	10.0.3.5	ACCEPT	DEFAULT_ACCEPT	
5:37:49 PM	TCP	255.89.145.169:1474	10.0.3.5	ACCEPT	DEFAULT_ACCEPT	
5:37:47 PM	TCP	244.37.214.222:1472	10.0.3.5	ACCEPT	DEFAULT_ACCEPT	
5:37:31 PM	ICMP	10.0.5.5	10.0.3.5 (Echo Reply)	DROP	RULE ENGINE WAN	
5:37:30 PM	ICMP	10.0.5.5	10.0.3.5 (Echo Reply)	DROP	RULE ENGINE WAN	
5:36:47 PM	TCP	151.220.188.2:1584	10.0.3.5	ACCEPT	DEFAULT_ACCEPT	
5:36:46 PM	TCP	135.190.140.76:1583	10.0.3.5	DROP	RULE ENGINE WAN	
5:36:45 PM	TCP	295.2.18.204:1582	10.0.3.5	DROP	RULE ENGINE WAN	
5:36:43 PM	TCP	81.152.69.228:1580	10.0.3.5	DROP	RULE ENGINE WAN	
5:36:42 PM	TCP	112.57.133.149:1579	10.0.3.5	DROP	RULE ENGINE WAN	

Figura 4.5: Pagina pentru jurnalizare

4.5 Evaluarea sistemului

Evaluarea sistemului CuciSec s-a desfășurat într-un mediu controlat compus din trei mașini virtuale, a căror topologie este ilustrată în Figura 4.6. Mașina *Atacator* (10.0.5.5) simulează sursa atacurilor și este legată de sistem prin interfața `enp0s8` (10.0.5.10), fiind partea untrusted. Mașina *Client* (10.0.3.5) simulează rețeaua protejată și este inclusă în interfața de rețea `enp0s9` (10.0.3.10). O a treia interfață, `enp0s2`, este utilizată exclusiv pentru management (accesul prin SSH la sistem pentru dezvoltare și mentenanță a codului, și administrarea rețelei cu aplicația web) fără a participa la traficul testat. Tranzitul pachetelor este activat pe VM-ul CuciSec, astfel încât tot traficul dintre atacator și rețeaua internă tranzitează obligatoriu lanțul forward al sistemului.

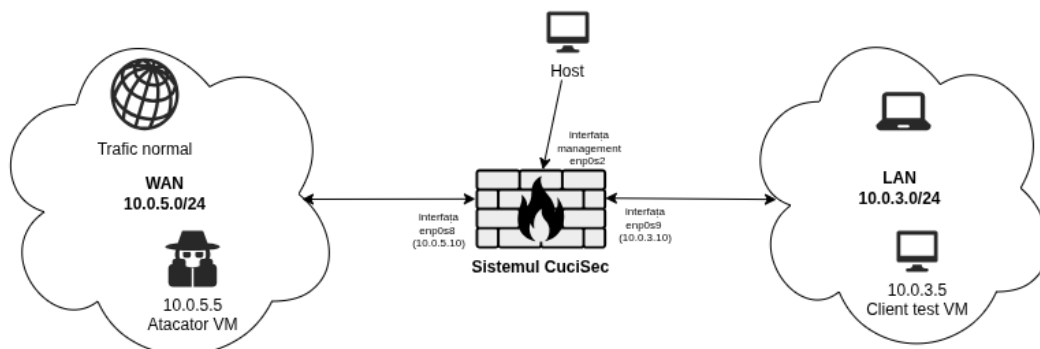


Figura 4.6: Mediul de testare

Instrumentele utilizate pentru testarea calității sistemului sunt: *hping3* pentru atacuri de volumetrice, *curl* pentru cereri HTTP cu payload malițios, *flag-uri* pentru diferite scenarii de test cum ar fi: *-rand-source*, *-spoof*, *-flood*; *nc* (netcat), *nmap* pentru scanare și conexiuni pe porturi capcană. Primele două scenarii validează comportamentul de bază al sistemului. Inițial verificăm validitatea conexiunii prin *ping-uri* ICMP generate de pe Atacator spre Client, care traversează pipeline-ul fără să corespundă niciunui tip de atac și este acceptat implicit, evenimentul fiind înregistrat în log-uri cu acțiunea `DEFAULT_ACCEPT`. O cerere HTTP fără payload malițios (`curl http://10.0.3.5`) către un server *Apache* pornit pe mașina client, trece prin DPIEngine fără să declanșeze nicio semnătură și primește același verdict, confirmând că traficul legitim nu este perturbat de prezența sistemului.

Scenariul trei validează protecția împotriva atacurilor DDoS generat cu comanda:

```
hping3 -S -p 80 --flood --rand-source 10.0.3.5
```

Atacul injectează pachete SYN cu adrese sursă generate aleator, simulând un atac distribuit. Contorul `global_syn_flood` din `nftables` crește rapid, fiind vizibil în dashboard în contorul aferent, confirmând că limitele globale absorb traficul înainte ca acesta să ajungă în userspace.

Scenariul patru utilizează aceeași comandă dar dintr-o singură sursă: 10.0.5.5. Inițial pachetele sunt limitate în kernel la un anumit număr pe secundă, apoi ajung în userspace și sunt numărate de FloodEngine pe parcursul a 12 secunde; după ce pragul de pachete este atins, adresa sursă este blocată și apare în tabelul din interfața web.

Scenariul următor validează capacitatea motorului Deep Packet Inspection de a detecta atacuri la nivelul aplicației, prin cereri malițioase către server sau o aplicație client. Comanda:

```
curl  
"http://10.0.3.5/?id=1%20UNION%20SELECT%20*%20FROM%20users"
```

trimite o cerere HTTP cu un payload SQL injection în care caracterele spațiu sunt codate ca `%20`. DPIEngine normalizează payload-ul înainte de potrivire, transformând `"1%20UNION%20SELECT"` în `"1 union select"`. Pachetul este eliminat și adresa IP sursa este interzisă. Câmpul `details` din log-ul generat conține valoarea `"DPI HIT: SQLi: UNION SELECT"`. Deasemena sistemul detectează și alte comenzi cu payload malițios:

- `curl "http://10.0.3.5/?user=admin'%20OR%20'1'='1'"`: reprezintă o injecție SQL de tip tautologie, menită să ocolească autentificarea forțând o condiție mereu adevărată;

- `curl "http://10.0.3.5:8000/?page=../../../../etc/passwd"`: încearcă un atac de tip *Path Traversal* pentru a accesa fișiere sensibile;
- `curl "http://10.0.3.5/?code=shell_exec%20('id')"`: simulează o vulnerabilitate de tip *Remote Code Execution (RCE)* care permite atacatorului să execute comenzi direct în sistemul de operare al serverului gazdă;
- `curl "http://10.0.3.5/?profile=<img%20src=x%20onerror=alert(1)>"`: simulează un atac de tip *Cross-Site Scripting (XSS)* prin injectarea unui tag `<script>`, posibile consecințe fiind furtul de sesiuni (session hijacking), redirectionarea utilizatorilor către site-uri malițioase sau defăimarea paginii web.
- `curl "http://10.0.3.5/" -A "sqlmap/1.5"`: sistemul blochează cererea trimisă de scannerul automat `sqlmap`, periculos prin capacitatea de a automatiza complet detectarea și exploatarea vulnerabilităților de tip *SQL injection*, ducând la compromiterea bazei de date.

Validitatea sistemului împotriva conexiunilor pe porturi cu risc ridicat este realizată prin comenzi de tipul `nc 10.0.3.5 4444`, sau care au în componență flag-ul `"-p"` împreună cu port vulnerabil: `sudo hping3 -S -p 23 --rand-source 10.0.3.5`

Scenariile de management validează funcționarea administrativă a sistemului. O regulă de tip *DROP* adăugată din interfața web pentru un port specific devine activă imediat, la următorul pachet care corespunde criteriilor, fără nicio întrerupere a procesării în curs. Deasemenea, operația de manual ban sau unban efectuată din interfața web blochează sau restaurează accesul adresei respective în toate straturile sistemului simultan, iar traficul din sursa respectivă traversează din nou pipeline-ul decizional pentru următorul pachet interceptat.

Din perspectiva performanței observate în cadrul scenariilor, sistemul menține responsivitatea interfeței web și a apelurilor API inclusiv în timpul atacurilor active, dashboard-ul actualizează statisticile și contoarele fără întârzieri, oricât de mare ar fi volumul de trafic interceptat și analizat.

Capitolul 5

Concluzii și Direcții Viitoare de Dezvoltare

5.1 Sinteza realizărilor și a contribuțiilor aduse

Sistemul CuciSec a pornit de la o problemă practică: utilitarele de filtrare a traficului disponibile pe Linux sunt eficiente la nivel de kernel, dar fără să ofere o logică decizională amănunțită și flexibilă, sau o interfață de administrare accesibilă. Obiectivul proiectului a fost construirea unui sistem care combină filtrarea rapidă cu inspecția avansată și monitorizarea vizuală a traficului, fără a compromite performanța sau siguranța rețelei. Separarea responsabilităților între nivelul kernel și cel userspace s-a dovedit a fi fundația sistemului fiind o decizie arhitecturală cel puțin necesară pentru asigurarea și menținerea traficului legitim.

Subsistemul de prevenire a intruziunilor, construit pe niveluri independente, a demonstrat că atacuri cu mecanisme complet diferite pot fi gestionate de același sistem, fără a introduce latență în fluxul de date.

Aplicația web a transformat sistemul dintr-un proces background într-un instrument de vizualizare și management a pachetelor. Administratorul inspectează în timp real evenimentele înregistrate, modifică regulile de filtrare și gestionează lista de adrese blocate fără nicio intervenție cu linia de comandă. Statisticile combinate din baza de date și din contoarele nftables oferă o imagine întreagă și persistentă asupra activității sistemului de-a lungul filtării traficului.

5.2 Limitări ale sistemului

Implementarea actuală a sistemului CuciSec acoperă scenariile de bază ale unui sistem firewall de generație următoare, demonstrând corectitudine și performanță în mediul de testare. Cu toate acestea, mai multe aspecte ale arhitecturii actuale repre-

zintă limitări care restrâng aplicabilitatea sistemului în contexte mai exigente sau mai variate.

O limitare arhitecturală considerabilă este procesarea secvențială a pachetelor prin funcția callback, care este invocată sincron pentru fiecare pachet, iar un singur fir de execuție parcurge în ordine toate etapele pipeline-ului decizional. La volume mari de trafic, timpul de execuție al funcției callback devine factorul de limită al debitului, indiferent de câte nuclee de procesor sunt disponibile pe mașina gazdă.

Inspecția payload-ului este limitată la traficul HTTP necriptat. Traficul HTTPS, care reprezintă cea mai mare parte a traficului web, nu este analizat: criptarea TLS împiedică inspecția conținutului. Semnăturile DPI reprezintă o a doua limitare de acoperire: expresiile regulate sunt definite static și nu se adaptează la variante noi ale atacurilor. Un atacator poate construi un payload care le ocolesc fără a fi oprit de niciun alt nivel al sistemului.

Testarea doar într-un mediu virtualizat cu trafic și pachete construite manual reprezintă o limitare importantă din punct de vedere funcțional, întrucât sistemul nu este testat în medii reale, cu pachete și fluxuri de date obișnuite. Atacurile simulate au fost executate într-un mediu controlat și izolat, având un caracter predictibil. În realitate, comportamentul atacatorilor este mult mai inteligent, fiind posibil ca sistemul actual să nu reușească în totalitate să intercepteze amenințările sofisticate.

5.3 Direcții viitoare de extindere

Sistemul CuciSec în forma sa actuală reprezintă o bază arhitecturală fundamentală a unui firewall de generație următoare, care poate fi extins și exploatat. Direcțiile de extindere identificate acoperă atât componente interne precum scalabilitatea sau inspecția cât și integrarea sistemului într-un ecosistem de securitate mai larg.

Cea mai semnificativă direcție de extindere este înlocuirea parțială sau completă a motorului DPI bazat pe expresii regulate statice cu un clasificator de tip Machine Learning antrenat pe seturi de date cu trafic posibil malițios și care ar putea identifica tipare de atac necunoscute anterior fără a necesita actualizarea manuală a semnăturilor. Implementarea unui astfel de clasificator în acest context ridică cerințe specifice: trebuie să se încadreze în constrângerile de latență ale buclei NFQUEUE: un model cu timp de inferență de ordinul milisecundelor per pachet este viabil, în timp ce un model complex care necesită sute de milisecunde ar bloca procesarea și ar reduce debitul sistemului la volume de trafic moderate. O abordare complementară este detecția anomaliilor prin metode nesupervizate, prin urmărirea comportamentului normal al surselor cunoscute și blocarea activităților suspecte.

O a doua direcție vizează inspecția pentru traficul HTTPS. Integrarea unui proxy de terminare TLS în fața sistemului ar oferi DPIEngine acces la payload-ul în clar

inclusiv pentru conexiunile criptate. O alternativă este inspecția la nivel de socket prin *Extended Berkeley Packet Filter* (eBPF), care poate accesa datele aplicației înainte de criptare, la nivelul stivei de rețea din kernel, fără un proxy intermediar.

Scalabilitatea procesării pachetelor poate fi îmbunătățită prin utilizarea mai multor cozi NFQUEUE, cu distribuirea traficului folosind un algoritm sau un tipar de clasificare. Fiecare coadă ar fi procesată de un proces separat, eliminând constrângerea thread-ului unic din implementarea actuală și valorificând toate nucleele de procesor disponibile.

Integrarea unui sistem de notificări reprezintă deasemenea o idee de extindere valoroasă, alertând administratorul pe telefon sau email la detectarea unei amenințări fără supravegherea continuă a dashboard-ului.

Fiecare dintre aceste direcții poate fi abordată independent, fără a necesita restructurarea arhitecturii actuale, maximizând potențialul sistemului.

Bibliografie

- [DH17] Dr. Steve E. Deering and Bob Hinden. Internet Protocol, Version 6 (IPv6) Specification. RFC 8200, July 2017. URL: <https://www.rfc-editor.org/info/rfc8200>, doi:10.17487/RFC8200.
- [ES01] Kjeld Borch Egevang and Pyda Srisuresh. Traditional IP Network Address Translator (Traditional NAT). RFC 3022, January 2001. URL: <https://www.rfc-editor.org/info/rfc3022>, doi:10.17487/RFC3022.
- [Eur25] European Union Agency for Cybersecurity (ENISA). ENISA threat landscape, 2025. Accesat: aprilie 2026. URL: <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2025>.
- [Fed25] Federal Bureau of Investigation. Salt Typhoon Cyber Espionage Campaign, 2025. Accesat: mai 2026. URL: <https://www.fbi.gov/investigate/cyber/alerts/2025/peoples-republic-of-china-cyber-threat-activity/>.
- [Goo24] Google Cloud. UNC5537 Targets Snowflake Customer Instances for Data Theft and Extortion, 2024. Accesat: mai 2026. URL: <https://cloud.google.com/blog/topics/threat-intelligence/unc5537-snowflake-data-theft-extortion>.
- [Goo26] Google Cloud. Cybersecurity Forecast, 2026. Accesat: mai 2026. URL: <https://cloud.google.com/security/resources/cybersecurity-forecast>.
- [Han14] Peter N.M. Hansteen. *The Book of PF: A No-Nonsense Guide to the OpenBSD Firewall*. No Starch Press, 3 edition, 2014. ISBN: 978-1-59327-589-1.
- [Kra25] David Krause. The \$1.4 Billion Bybit Hack: Cybersecurity Failures and the Risks of Cryptocurrency Deregulation. 2025. Accesat: mai 2026. doi: 10.51483/IJCCR.5.1.2025.52-62.
- [Mic24a] Microsoft Corporation. Roadmap for Developing WFP Callout Drivers, 2024. Accesat: mai 2026. URL: <https://learn.microsoft.com/en>

-us/windows-hardware/drivers/network/roadmap-for-developing-wfp-callout-drivers.

- [Mic24b] Microsoft Corporation. Windows Filtering Platform, 2024. Accesat: mai 2026. URL: <https://learn.microsoft.com/en-us/windows/win32/fwp/windows-filtering-platform-start-page>.
- [Nat26] National Institute of Standards and Technology. Resource Center Glossary, 2026. Accesat: aprilie 2026. URL: <https://csrc.nist.gov/glossary>.
- [Pos80] Jon Postel. User Datagram Protocol. RFC 768, IETF, 1980. Accesat: aprilie 2026. URL: <https://www.rfc-editor.org/info/rfc768>.
- [Pos81a] Jon Postel. Internet Protocol. RFC 791, IETF, 1981. Accesat: aprilie 2026. URL: <https://www.rfc-editor.org/info/rfc791>.
- [Pos81b] Jon Postel. Transmission Control Protocol. RFC 793, IETF, 1981. Accesat: aprilie 2026. URL: <https://www.rfceditor.org/info/rfc793>.
- [Pow25] PowerSchool. PowerSchool Cybersecurity Incident Customer FAQ, 2025. Accesat: mai 2026. URL: <https://www.pcmag.com/news/powerschool-confirms-breach-was-your-kids-data-exposed>.
- [PyP23] PyPI Contributors. NetfilterQueue Python bindings for libnetfilter_queue, 2023. Accesat: aprilie 2026. URL: <https://pypi.org/project/NetfilterQueue/>.
- [Ras07] Michael Rash. *Linux Firewalls: Attack Detection and Response with iptables, psad, and fwsnort*. No Starch Press, 2007. ISBN: 978-1-59327-141-1.
- [Rus00] Paul Russell. Linux IPCHAINS-HOWTO, 2000. Accesat: mai 2026. URL: <https://tldp.org/HOWTO/IPCHAINS-HOWTO.html>.
- [SH09] Karen Scarfone and Paul Hoffman. Guidelines on Firewalls and Firewall Policy. Special Publication 800-41 Revision 1, National Institute of Standards and Technology (NIST), 2009. Accesat: aprilie 2026. URL: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-41r1.pdf>.
- [Spi02] Lance Spitzner. *Honeypots: Tracking Hackers*. Addison-Wesley Professional, 2002. ISBN: 978-0-32-110895-1.
- [Sta17] William Stallings. *Network Security Essentials: Applications and Standards*. Pearson, 6 edition, 2017. ISBN: 978-1-292-15485-5.

- [Sue15] Steve Suehring. *Linux Firewalls: Enhancing Security with nftables and Beyond*. Pearson Education, 4 edition, 2015. ISBN: 978-0-13-400002-2.
- [TAD08] AbuHmed Tamer, Mohaisen Abedelaziz, and Nyang DaeHun. A Survey on Deep Packet Inspection for Intrusion Detection Systems, 2008. Accesat: mai 2026. URL: <https://arxiv.org/pdf/0803.0037>, arXiv: 0803.0037.
- [Thea] The Netfilter Project. libnetfilter_queue Documentation. Accesat: aprilie 2026. URL: https://netfilter.org/projects/libnetfilter_queue/doxygen/html/.
- [Theb] The Netfilter Project. nftables Official Documentation. Accesat: aprilie 2026. URL: <https://www.netfilter.org/projects/nftables/index.html>.
- [TW10] Andrew S. Tanenbaum and David J. Wetherall. *Computer Networks*. Pearson Prentice Hall, 5 edition, 2010. ISBN: 978-0-13-212695-3.
- [U.S25] U.S. Department of Health and Human Services. Change Healthcare Cybersecurity Incident FAQs, 2025. Accesat: mai 2026. URL: <https://www.aha.org/system/files/media/file/2025/02/Change-Healthcare-Cyberattack-Underscores-Urgent-Need-to-Strengthen-Cyber-Preparedness.pdf>.
- [Wor26] World Economic Forum. Global Cybersecurity Outlook 2026, January 2026. Accesat: aprilie 2026. URL: <https://www.weforum.org/publications/global-cybersecurity-outlook-2026/>.